

HTML 5

&

CSS 3

Création de sites Web

Objectifs

Nous découvrirons dans ce cours les célèbres langages HTML5 et CSS3 que l'on utilise aujourd'hui pour **concevoir des sites web**. Vous découvrirez notamment comment :

- Insérer du texte, des images et des vidéos
- Faire des liens entre vos pages
- Mettre en forme en modifiant la couleur, la taille, le fond, la police...
- Positionner les éléments du site comme bon vous semble : en-tête, menus...
- et bien plus encore !

Pourquoi deux langages ? Pour simplifier et organiser les choses ! Le HTML définit le contenu, le CSS permet, lui, de définir la présentation : couleurs, image de fond, marges, taille du texte...

Un site internet codé uniquement en HTML/CSS est appelé site statique, aucune interaction n'est possible avec le visiteur. Parfois d'autres langages comme le PHP seront nécessaires. Le PHP permet, par exemple, l'envoi et le traitement d'un formulaire, mais aussi l'accès à une base de données pour créer dynamiquement des pages, par exemple...

Historique

Voici un très bref historique de ces langages pour votre culture générale.

Les versions de HTML

HTML (HyperText Markup Language) : il a fait son apparition dès 1991 lors du lancement du Web. Son rôle est de gérer et organiser le contenu. C'est donc en HTML que vous écrirez ce qui doit être affiché sur la page : du texte, des liens, des images... Vous direz par exemple : « Ceci est mon titre, ceci est mon menu, voici le texte principal de la page, voici une image à afficher, etc. ».

HTML 1 : c'est la toute première version créée par Tim Berners-Lee en 1991.

HTML 2 : la deuxième version du HTML apparaît en 1994 et prend fin en 1996 avec l'apparition du HTML 3.0. C'est cette version qui posera en fait les bases des versions suivantes du HTML. Les règles et le fonctionnement de cette version sont donnés par le W3C (tandis que la première version a été créée par un seul homme).

HTML 3 : apparue en 1996, cette nouvelle version du HTML rajoute de nombreuses possibilités au langage comme les tableaux, les applets, les scripts, le positionnement du texte autour des images, etc.

HTML 4 : il s'agit de la version la plus répandue du HTML (plus précisément, il s'agit de HTML 4.01). Elle apparaît pour la première fois en 1998 et propose l'utilisation de frames (qui découpent une page web en plusieurs parties), des tableaux plus complexes, des améliorations sur les formulaires, etc. Mais surtout, cette version permet pour la première fois d'exploiter des feuilles de style, notre fameux CSS !

HTML 5 : c'est LA dernière version. Elle apporte de nombreuses améliorations comme la possibilité d'inclure facilement des vidéos, un meilleur agencement du contenu, de nouvelles fonctionnalités pour les formulaires, etc. C'est cette version que nous allons découvrir ensemble.

Les versions de CSS

CSS (Cascading Style Sheets, aussi appelées Feuilles de style) : le rôle du CSS est de gérer l'apparence de la page web (agencement, positionnement, décoration, couleurs, taille du texte...). Ce langage est venu compléter le HTML en 1996.

CSS 1 : dès 1996, on dispose de la première version du CSS. Elle pose les bases de ce langage qui permet de présenter sa page web, comme les couleurs, les marges, les polices de caractères, etc.

CSS 2 : apparue en 1999 puis complétée par CSS 2.1, cette nouvelle version de CSS rajoute de nombreuses options. On peut désormais utiliser des techniques de positionnement très précises, qui nous permettent d'afficher des éléments où on le souhaite sur la page.

CSS 3 : c'est la dernière version, qui apporte des fonctionnalités particulièrement attendues comme les bordures arrondies, les dégradés, les ombres, etc.

Quels logiciels

S'il existe des logiciels très complexes et wysiwyg (What You See Is What You Get), il suffit de deux logiciels courants et gratuits pour créer un site internet :

- Un éditeur de texte pour écrire le code
- Un navigateur internet pour voir le résultat

Dans le cadre de ce cours, vous utiliserez Notepad++. Il est déjà installé sur les ordinateurs du lycée.

Un avantage d'utiliser un tel logiciel est, par exemple, la coloration des mots-clés. Vous voyez ainsi instantanément si les mots-clés utilisés sont bien orthographiés ou non... A la première utilisation, il faudra, cependant lui indiquer que vous écrivez du HTML. Ça se passe dans le menu Langages / H / HTML. Après enregistrement, ça se fera tout seul ! A condition de ne pas oublier d'indiquer l'extension du fichier !

Premiers principes de HTML

HTML est un langage de balises, c'est à dire qu'on indique (le plus souvent) chaque élément sous la forme :

```
<balise>
  contenu...
</balise>
```

En fait, il y a aussi quelques balises qui s'écriront sous la forme :

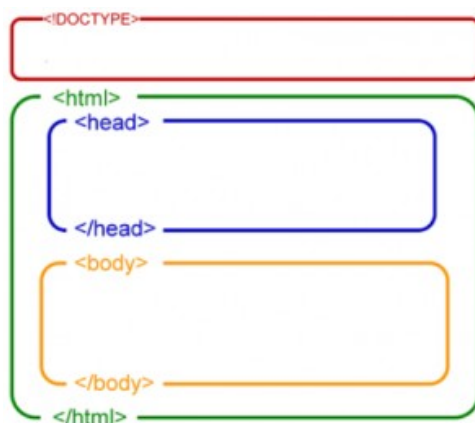
```
<balise contenu... />
```

On les appelle parfois des balises orphelines !

En HTML, tout est structuré en balises. Des balises peuvent contenir d'autres balises, mais il faut faire attention à l'ordre de fermeture ! La dernière balise ouverte doit être la première à être fermée. Pour cela le plus simple est de respecter une indentation, c'est à dire une décalage de chaque élément vers la droite...

Correct	Incorrect
<pre><balise1> <balise2> contenu... </balise2> </balise1></pre>	<pre><balise1> <balise2> contenu... </balise1> </balise2></pre>

Mais voyons maintenant les balises définissant la structure du fichier.



Le doctype

Cette balise est un peu particulière, c'est elle qui va définir la version de HTML utilisée... Pendant longtemps, c'était très compliqué... Pour XHTML 1.0, il fallait écrire :

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

Dans le cadre de HTML5, il a été décidé de la simplifier, pour le plus grand bonheur des webmasters. Quand vous voyez une balise :

```
<!DOCTYPE html>
```

Cela signifie que la page est écrite en HTML5.

La balise <HTML>

C'est la balise principale du code. Elle englobe toute votre page. Comme vous pouvez le voir, la balise fermante </html> se trouve tout à la fin du code ! Dans cette balise, on trouve deux autres balises structurantes <head> et <body>

Une page web est en effet constituée de deux parties :

- **L'en-tête <head>** : cette section donne quelques informations générales sur la page comme son titre, l'encodage (pour la gestion des caractères spéciaux), etc. Cette section est généralement assez courte, plus courte que la section body, en tous les cas !

- Les informations que contient l'en-tête ne sont pas affichées sur la page, ce sont simplement des informations générales à destination de l'ordinateur. Elles sont cependant très importantes !
- **Le corps <body>** : c'est là que se trouve la partie principale de la page. Tout le texte que nous écrivons ici sera affiché à l'écran. C'est à l'intérieur du corps que nous écrivons la majeure partie de notre code.

Une dernière chose sur cette balise **<html>** est qu'il est recommandé d'y ajouter un attribut de langue. En effet si le site est entièrement écrit pour une seule langue, on mettra :

```
<html lang="fr"
```

pour définir le texte en français. Cet attribut de langue est universel, c'est à dire qu'il peut être ajouté sur n'importe quelle balise (nous reviendrons bientôt sur ces attributs). Elle a plusieurs utilités. Que ce soit pour la traduction des pages, le choix d'une police ou la lecture des écrans à destination des aveugles ou mal-voyants, cet attribut est fortement recommandé par le W3C !

Une première page

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <title>Ma page</title>
  </head>
  <body>
    Ma première page
    Où j'écris ce que je veux...
  </body>
</html>
```

Par ordre « d'appartenance », on décale les balises. Ce n'est pas obligatoire et cela n'a aucun impact sur l'affichage de la page, mais cela rend le code source plus lisible. On appelle cela **l'indentation**. Dans votre éditeur, il suffit d'appuyer sur la touche Tab pour avoir le même résultat.

Le charset

```
<meta charset="utf-8" />
```

Cette balise indique l'encodage utilisé dans votre fichier. Il faut aussi que votre fichier soit bien enregistré en UTF-8. C'est le cas le plus souvent sous Linux par défaut mais, sous Windows, il faut généralement le dire au logiciel.

Sous Notepad++, allez dans le menu *Encodage > Encoder en UTF-8 (sans BOM)*. Cela ne s'applique qu'au fichier actuellement ouvert. Pour ne pas avoir à le faire pour chaque nouveau fichier, je vous conseille d'aller dans le menu *Paramétrage > Préférences, onglet Nouveau document/Dossier*. Sélectionnez *UTF-8 sans BOM* dans la liste.

Si vous rencontrez des problèmes d'affichage des accents, c'est probablement une question d'encodage...

La balise title

C'est le titre de votre page, un des éléments les plus importants ! Toute page doit avoir un titre qui décrit ce qu'elle contient. Il est conseillé de garder le titre assez court (moins de 100 caractères en général).

Un peu plus loin dans le code...

Une balise c'est bien, mais si en plus, on peut affiner son comportement, c'est mieux !

Des attributs

Prenons l'exemple d'une balise qui serait destinée à afficher une image. C'est une balise orpheline.

```
<image />
```

Tel quel, ça ne sert à rien ! Quel est, par exemple, le nom du fichier contenant l'image à afficher ? Pour remédier à ce manque, il faut ajouter des attributs.

```
<image source="photo.jpg" />
```

source est un attribut de la balise image, c'est à dire un élément qui va apporter une information complémentaire à la balise.

Pour le cas où ce n'est pas une balise orpheline, on ajoute les attributs uniquement dans la balise ouvrante.

```
<balise attribut="valeur">
    contenu...
</balise>
```

Par exemple :

```
<citation auteur="Neil Armstrong" date="21/07/1969">
    C'est un petit pas pour l'homme, mais un bond de géant pour l'humanité.
</citation>
```

Malheureusement ce ne sont pas de vraies balises ! Les noms véritables sont en anglais.

Des commentaires

Dans tous les langages de programmation, il existe une façon de mettre des commentaires. Ils ne sont pas affichés et n'ont généralement aucune incidence sur le fonctionnement du code, mais ils ne sont pas pour autant inutiles ! Ils permettent de donner des indications au lecteur (ou à vous même quand vous revenez sur un ancien code) à propos de ce qui a été fait.

En HTML, les commentaires s'écrivent de la façon suivante...

```
<!-- Ceci est un commentaire -->
```

Par exemple, si on reprend notre première page, on pourra écrire :

```
<!DOCTYPE html>
<html>
  <head>
    <!-- En-tête de la page -->
    <meta charset="utf-8" />
    <title>Titre</title>
  </head>
  <body>
    <!-- Corps de la page -->
    Première page avec quelques commentaires !
    Ceux-là ne sont pas forcément essentiels, mais c'est pour l'exemple !
  </body>
</html>
```

Organiser son texte

Dans votre première page, vous pouvez bien taper tout le texte que vous voulez, même avec des retours à la ligne, tout sera affiché à la queue-leu-leu ! Ce navigateur, il faut vraiment tout lui dire... Même de revenir à la ligne ! En fait d'un navigateur à un autre, d'un ordinateur à un autre, les polices et les largeurs d'écrans changent... Ça peut donc vite s'avérer impossible de mettre les retours à la ligne au bon endroit... Autant laisser le navigateur s'en occuper !

Aller à la ligne

Il existe une balise pour cela ! Cela permet d'écrire le texte sous la forme qu'on souhaite au niveau du code, tout en donnant d'autres indications sur la forme que ça devra avoir dans le navigateur ! Pour aller à la ligne, il s'agit de la balise

```
<br />    ou    <br>
```

Cette balise orpheline (break en anglais) permet donc de forcer le navigateur à revenir à la ligne. Notez que le slash final, s'il était obligatoire en XHTML, ne l'est plus du tout... C'est comme vous voulez !

Si on reprend notre page de tout à l'heure...

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <!-- En-tête de la page -->
    <meta charset="utf-8" />
    <title>Titre</title>
  </head>
  <body>
    <!-- Corps de la page -->
    Première page avec quelques commentaires !<br />
    Ceux-là ne sont pas forcément essentiels, mais c'est pour l'exemple !
  </body>
</html>
```


On a maintenant un retour à la ligne et notre texte est donc écrit sur deux lignes au lieu d'une. C'est un peu moins dense et un peu plus lisible !

Définir des paragraphes

Dans une certaine mesure, c'est un autre moyen de revenir à la ligne... La balise

```
<p>
  texte...
</p>
```

permet d'encadrer un paragraphe. Ce n'est cette fois pas une balise orpheline et il faudra penser à la balise fermante. Ce n'est pas simplement un retour à la ligne. Ça dit au navigateur qu'il s'agit d'un bloc de texte. Nous verrons bientôt (en CSS) qu'il est simple de dire au navigateur de mettre tous les blocs de texte en rouge ou bleu, gras ou italique ou les deux, avec des marges ou toute autre mise forme imaginable !

Rien n'empêche de mettre des retours à la ligne `<br />` dans une balise `<p>` ! En fait, dans la mesure où à peu près tout le texte doit se trouver dans des balises `<p>`, les balises `<br />` seront même, idéalement, toujours dans des balises `<p>` !

```
<html>
  <head>
    <meta charset="utf-8" />
    <title>Sauts de ligne</title>
  </head>
  <body>
    <p>
      Bonjour et bienvenue sur mon site !<br />
      Ceci est mon premier test alors soyez indulgents s'il vous
      plaît, j'apprends petit à petit comment cela marche.
    </p>
    <p>
      Pour l'instant c'est un peu vide, mais revenez dans 2-3
      jours quand j'aurai appris un peu plus de choses, je vous assure que
      vous allez être surpris !
    </p>
  </body>
</html>
```

Indiquer des titres

Lorsque le contenu de votre page va s'étoffer avec de nombreux paragraphes, il va devenir difficile pour vos visiteurs de se repérer. C'est là que les titres deviennent utiles.

En HTML, on peut utiliser six niveaux de titres différents. On a donc six balises de titres différentes :

```
<h1> Titre très important </h1>
<h2>Titre un peu moins important </h2>
<h3> Titre encore moins important </h3>
et ainsi de suite jusqu'à <h6> </h6> qui n'est vraiment pas important du tout !
```

Au début, on a tendance à négliger ces titres... C'est une erreur. Les moteurs de recherches comme google et les autres utilisent ces balises pour le référencement afin de définir ce qu'il faut indexer de façon importante ou pas. Ça conditionnera donc la mise en avant de certaines parties de votre site !

```
<html>
  <head>
    <meta charset="utf-8" />
    <title>Sauts de ligne</title>
  </head>
  <body>
    <h1>Bonjour et bienvenue sur mon site !</h1>
    <p>
      Ceci est mon premier test alors soyez indulgents s'il vous
      plaît, j'apprends petit à petit comment cela marche.
    </p>
    <h2> je progresse...</h2>
    <p>
      Pour l'instant c'est un peu vide, mais revenez dans 2-3
      jours quand j'aurai appris un peu plus de choses, je vous assure que
      vous allez être surpris !
    </p>
  </body>
</html>
```

Mettre en valeur certains mots

Au sein de vos paragraphes, certains mots sont parfois plus importants que d'autres et vous aimeriez les faire ressortir. HTML vous propose différents moyens de mettre en valeur le texte de votre page.

Mettre un peu en valeur

Pour mettre un peu en valeur votre texte, vous devez utiliser la balise `<em> ... </em>`.

Son utilisation est très simple : encadrez les mots à mettre en valeur avec ces balises et c'est tout ! Par défaut, le plus souvent, ça met *le texte en italique*, mais CSS permettra de changer ce comportement.

Mettre beaucoup en valeur

Pour indiquer au navigateur que certains sont très importants, vous devez utiliser la balise `<strong> ... </strong>`. Là encore, encadrez les mots à mettre en valeur avec ces balises et c'est tout ! Par défaut, ça met **le texte en gras** mais ce n'est pas obligatoire.

Marquer du texte

Il existe aussi la balise `<mark> </mark>` qui permet de détacher un morceau de texte du reste sans pour autant dire qu'il est plus important. Cela aura une influence sur le référencement de votre site. Par défaut, **le texte sera surligné**.

Attention !

Au début on a tendance à penser que `<em>` met en italique, `<strong>` en gras et que `<mark>` surligne le texte. C'est souvent vrai, mais ce n'est pas leur but !!!

Le rôle de ces balises est d'indiquer le sens, la valeur du texte. Ainsi, `<strong>` indique à l'ordinateur « **Ce texte est important** ». C'est tout. Pour montrer que le texte est important, l'ordinateur décide de le mettre en gras (mais il pourrait aussi bien l'écrire en rouge !).

La plupart des navigateurs affichent les textes importants en gras, mais rien ne les y oblige.

Des listes et des listes

Dans un CV, on énumère des trucs. Ca serait bien de prévoir une balise pour dire à l'ordinateur que tout ce qui suit est une liste de quelque chose... La liste des ingrédients pour faire un délicieux guacamole, par exemple... Et puis la liste des opérations à effectuer pour bien le réussir !

Cela dit, il y a une différence entre les deux... Dans le premier cas, l'ordre importe peu, c'est juste une liste, si on mélange les éléments c'est pareil... c'est une liste non-ordonnée. Dans le second cas, il faut faire les choses dans l'ordre, sinon, ça ne marche pas... C'est une liste ordonnée !

Pour information, il existe un troisième type de liste, beaucoup plus rare : la liste de définitions. Elle fait intervenir les balises `<dl>` (pour délimiter la liste), `<dt>` (pour délimiter un terme) et `<dd>` (pour délimiter la définition de ce terme). Je n'en parlerai pas trop ici car, à moins d'écrire un dictionnaire, les deux autres sont de très loin les plus courantes !! Vous chercherez sur Google, si vous êtes curieux...

Les listes non-ordonnées

Comme leur nom l'indique, les listes non-ordonnées n'ont pas de notion d'ordre et on peut mélanger leurs éléments sans que cela modifie leur sens. Une liste de courses est un bon exemple de liste non-ordonnée (l'ordre dans lequel on achète les produits n'a pas grande importance).

Les listes non-ordonnées apparaissent parfois à des endroits inattendus : les barres de menus sont par exemple des listes non-ordonnées auxquelles on a appliqué quelques règles CSS pour la mise en forme.

Dans ce cas, c'est la balise `<ul>` qui sera utilisée (unordered list). Chaque élément sera indiqué dadans dans une balise `<li>`.

```
<ul>
  <li>Un élément</li>
  <li>un autre élément</li>
  <li>etc.</li>
</ul>
```

Les listes ordonnées

Dans ce cas, les éléments de la liste doivent être lus ou réalisés dans un ordre donné, les éléments sont donc numérotés. On retrouve souvent ces listes pour décrire des processus pas-à-pas (comme un mode d'emploi ou une recette de guacamole) où chaque étape ne peut être réalisée que si l'action précédente a été traitée.

Ici, c'est la balise `<ol>` qui sera utilisée (ordered list). Chaque élément sera, là encore, indiqué dadans dans une balise `<li>`.

```
<ol>
  <li>Elément 1</li>
  <li>Elément 2</li>
  <li>etc.</li>
</ol>
```

Et le guacamole alors ?

```
<h1>Guacamole rapide à faire</h1>

<h2>Ingrédients</h2>
<ul>
  <li>2 avocats (pelés et avec les noyaux retirés)</li>
  <li>le jus d'un citron</li>
  <li>1/4 de concombre, coupé grossièrement</li>
  <li>1 petite tomate, coupée</li>
</ul>

<h2>Instructions</h2>
<ol>
  <li>Écrasez délicatement les avocats avec une fourchette</li>
  <li>Placez la purée obtenue dans un plat et arrosez avec le jus de citron</li>
  <li>Mélangez pour que le jus de citron empêche la purée d'avocat de
noircir</li>
  <li>Mélangez la tomate et le concombre coupés</li>
  <li>Gardez au frais et servir rapidement avec des tortillas</li>
</ol>
```

donnera dans le navigateur :

Guacamole rapide à faire

Ingrédients

- 2 avocats (pelés et avec les noyaux retirés)
- le jus d'un citron
- 1/4 de concombre, coupé grossièrement
- 1 petite tomate, coupée

Instructions

1. Écrasez délicatement les avocats avec une fourchette
2. Placez la purée obtenue dans un plat et arrosez avec le jus de citron
3. Mélangez pour que le jus de citron empêche la purée d'avocat de noircir
4. Mélangez la tomate et le concombre coupés
5. Gardez au frais et servir rapidement avec des tortillas

Des attributs ?

Il y a plusieurs attributs historiques dont je ne parlerai pas parce qu'ils sont dépréciés et ne seront probablement bientôt plus supportés par les navigateurs. Dans vos recherches sur google, vous pourrez cependant les croiser... ne les utilisez pas. Ils sont tous avantageusement remplacés par des directives CSS qui seront données dans un fichier séparé. **N'oubliez pas, le HTML pour le contenu, le CSS pour la forme...**

Hormis les attributs universels (sur lesquels nous reviendrons), il y a cependant deux attributs utilisables avec les listes ordonnées `<ol>`.

L'attribut start

Permet de définir à quel numéro on commence. On n'est pas forcé de commencer au 1 ! Ça permet de continuer une liste qui a été commencée plus haut...

```
<ol start="3">
  <li> le premier élément a en fait le numéro 3</li>
  <li> puis le numéro 4 </li>
</ol>
```

L'attribut reversed

Permet de mettre les numéros dans l'ordre décroissant. C'est un attribut booléen, sa simple présence le rend actif, il n'y a pas besoin d'ajouter une valeur.

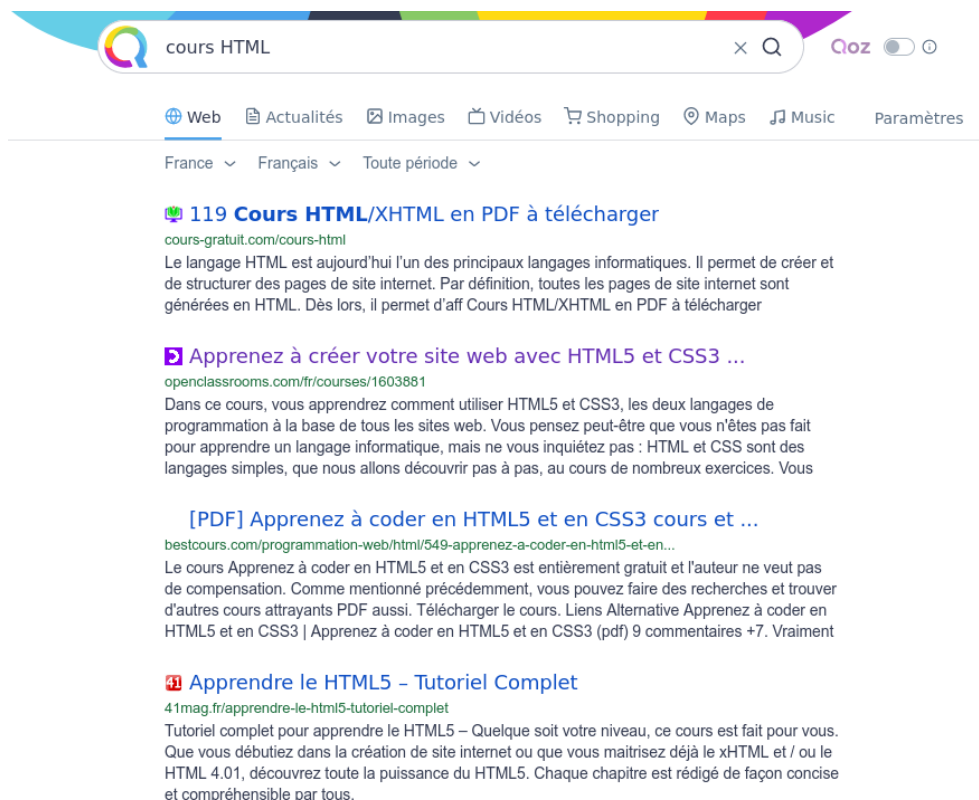
```
<ol start="3" reversed>
  <li> le premier élément a encore le numéro 3</li>
  <li> puis le numéro 2 </li>
</ol>
```

Des listes de listes...

Rien n'empêche de faire des listes de listes. Pour des menus à plusieurs niveaux, par exemple, c'est très fréquent...

Des liens

Un site Web est généralement composé de plusieurs pages, il faut pouvoir passer aisément d'une page à une autre, mais vous avez l'habitude de ce fonctionnement, il s'agit des liens hyper-texte. N'importe quel moteur de recherche renvoie des pages et des pages de liens correspondant plus ou moins bien à vos recherches !



Pour faire ces liens, on utilise la balise `<a>` (pour anchor, en anglais). Elle s'écrit de la façon suivante :

```
<a>texte du lien</a>
```

Les attributs

La balise, a elle toute seule ne sert pas à grand-chose, elle ne dit même pas où il faut aller... Les attributs sont là pour ça.

L'attribut href

C'est celui qui, précisément, dit où il faut aller quand on clique dessus. Par exemple, si le site a une deuxième page située dans le même dossier :

```
<a href="page2.html">Aller à la page 2</a>
```

Quand vous ferez votre barre de menu, vous écrirez une liste de liens vers les différentes pages de votre site.

En pratique *href* permet de mettre l'url de la page ou vous voulez aller. Ce chemin peut être relatif ou absolu. Rien n'empêche, par exemple, d'écrire :

```
<a href="https://www.stcharles-stecroix.org/">Lien vers mon lycée</a>
```

L'attribut target

Lui, définit la cible du lien, c'est à dire comment va s'ouvrir le lien quand on va cliquer dessus. Par défaut, la nouvelle page s'ouvrira à la place de l'ancienne, mais on peut définir un comportement différent.

Valeur	Description
_blank	Ouvre la page liée dans une nouvelle fenêtre
_self	Ouvre le document lié dans le même cadre (frame), où il a été activé (comportement par défaut)
_parent	Ouvre le document lié dans le frame parent
_top	Ouvre le document lié dans le corps entier de la fenêtre
framename	Ouvre le document lié dans un frame nommé

_self, c'est le comportement par défaut, le nouveau contenu remplace l'ancien... Vous serez surtout amenés à utiliser *_blank* qui vous permettra d'ouvrir la page indiquée dans un nouvel onglet du navigateur (ou dans une nouvelle fenêtre si votre navigateur n'a pas d'onglets).

```
<a href="https://www.stcharles-stecroix.org/" target="_blank">Lien vers mon lycée</a>
```

Pour les autres valeurs, ça concerne les frames et vous ne devriez pas en avoir besoin ! Sachez juste que ça existe...

L'attribut title

Cet attribut permet d'afficher une bulle d'information quand on passe la souris sur le lien.

```
<a href="https://www.stcharles-stecroix.org/" title="St-Charles">Lien vers mon lycée</a>
```

D'une page vers... la même page !

Parfois les pages sont très longues. Ce sont les sites dits "one-page". Les différents contenus sont alors accessibles en scrollant ou en utilisant des liens !

Un attribut universel essentiel : id

Les attributs universels sont des attributs qui peuvent être appliqués sur TOUS les éléments. Les attributs que nous avons vus jusqu'à présent sont spécifiques à certaines balises. L'attribut *id* peut **toujours** être appliqué, quelle que soit la balise.

Cet attribut permet de donner un nom unique à la balise, un **identifiant**. Cet identifiant pourra servir ensuite pour donner des règles CSS à l'élément considéré et seulement à lui...

Dans une page, deux éléments ne peuvent pas avoir le même attribut *id*. C'est un nom unique que l'on choisira de façon à le représenter au mieux !

```
<h1 id="titre-principal">Mon premier site</h1>
<ul id="liste-ingredients">
  <li>...</li>
  <li>...</li>
</ul>
```

Et le lien alors ?

Eh bien, c'est simple, il suffit de faire un lien en mettant le nom de l'*id* précédé d'un caractère # dans l'attribut *href*. Par exemple, pour aller au titre principal que nous avons défini dans l'exemple :

```
<a href="#titre-principal">Aller au grand titre</a>
```

Des images

Dans tous les sites internet (ou presque), il y a des images ! Pourquoi ? Parce que c'est joli... Mais ça peut aussi être utile. Pourquoi pas une photo sur un CV...

Insertion d'une image

Alors, quelle est cette balise qui va nous permettre d'insérer une image ? Il s'agit de... `<img />` !

C'est une balise orpheline (comme `<br />`). Cela veut dire qu'on n'a pas besoin de balise fermante. En effet, nous n'avons pas besoin de délimiter quoique ce soit, nous voulons juste insérer une image à un endroit précis.

Une image est entre autres définie par un nom : *paysage_de_neige.jpg* par exemple. Nous reviendrons sur les différents formats d'image et la compression associée.

Choisissez bien le nom de votre image ! Si vous voulez éviter des problèmes, prenez l'habitude d'enregistrer vos fichiers avec des noms **en minuscules, sans espace ni accent**. Sous Windows, ça ne posera pas beaucoup de problèmes, mais la plupart des serveurs Web sont sous Linux et là, c'est un peu plus rigoureux !

Mais revenons à notre balise... Par défaut, elle ne peut pas savoir quelle image il faut afficher ! Il va falloir passer par les attributs.

L'attribut src

Il permet d'indiquer où se trouve l'image que l'on veut insérer.

Vous pouvez soit mettre un chemin absolu (<http://www.site.com/fleur.png>), soit, et c'est ce qu'on fait le plus souvent, mettre le chemin en relatif. Ainsi, si votre image est dans un sous-dossier *images*, vous devrez indiquer

```

```

L'attribut alt

Cela signifie « texte alternatif ». On doit **toujours** indiquer un texte alternatif à l'image, c'est-à-dire un court texte qui décrit ce que contient l'image. Ce texte sera affiché à la place de l'image si celle-ci ne peut pas être affichée ou téléchargée ou dans les navigateurs de personnes handicapées (non-voyants) qui ne peuvent malheureusement pas « voir » l'image. Cela aide aussi les robots des moteurs de recherche pour les recherches d'images.

Pour la fleur, on mettrait par exemple :

```

```

L'attribut title

Lui, il a la même fonction que pour la balise `<a>`, c'est à dire d'afficher une info-bulle quand on passe la souris sur l'image.

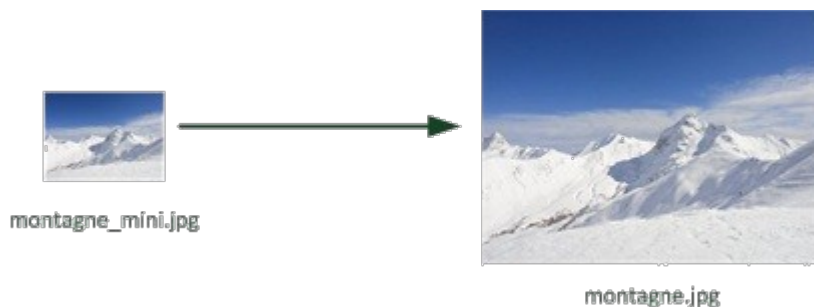
```

```

Un lien sur une image

On veut parfois rendre une image cliquable pour afficher l'image en plus grand, par exemple, ou encore accéder à une autre page. Ça se fait, sans surprise, avec une balise `<a>` ! Par exemple, si nos images miniature et grande sont dans le dossier *img*, on écrira :

```
<a href="img/montagne.jpg">  
    
</a>
```



Quand l'utilisateur va cliquer sur l'image miniature, le navigateur va directement afficher l'image en grand. On peut bien entendu utiliser les différents attributs propres à la balise `<a>`, puisque C'EST une balise `<a>` !

Des figures

En HTML5, on a en plus la balise `<figure>`. Elle a un rôle avant tout sémantique. Cela veut dire qu'elle indique à l'ordinateur que l'image a du sens et qu'elle est importante pour la bonne compréhension du texte. Cela peut aider au référencement des images et au référencement du site lui-même.

Des figures, donc...

Cette balise permet d'encadrer une ou plusieurs images et éventuellement de leur donner une légende en utilisant `<figcaption>`.

```
<figure>
  
  
  
  <figcaption>Logos des différents navigateurs</figcaption>
</figure>
```

En route vers le CSS...

Pour rendre vos pages plus jolies, on va enfin attaquer le CSS. Il y aura encore beaucoup de choses à apprendre sur le HTML, mais déjà vous avez de la matière à mettre en forme.

A quoi ça sert ?

Pendant longtemps, le CSS n'existait pas. La mise en forme et le contenu étaient mis dans le fichier HTML. Nous verrons qu'on peut toujours le faire mais que c'est fortement déconseillé et j'expliquerai pourquoi !

Pour l'instant, et je ne le répéterai jamais assez, ne mélangez pas le fond et la forme. Le fond c'est le texte, la forme c'est la taille, la couleur, les bordures, l'arrière plan, etc. Tout cela sera dans le CSS. Mais aussi il contiendra toute la mise en page. Il va vite devenir simple de définir des marges, de dire que tel bloc est toujours visible ou aligné à gauche sur 250 pixels de large...

D'un navigateur à l'autre

Celui qui fait tout le travail d'affichage, c'est le navigateur internet. Il prend vos fichiers, tache de comprendre ce qu'ils contiennent et affiche ce qu'il pense avoir compris. Mais si ça marchait tout le temps, ça serait trop simple !

En effet, il y a beaucoup de navigateurs différents, qui ne comprennent pas toujours les choses de la même façon... Parfois, même, qui ne les comprennent pas du tout ! Alors, ils font ce qu'ils peuvent pour faire au mieux... Pour les commandes de base, sur des navigateurs récents, en général, ça se passe bien. Mais il reste des navigateurs anciens, il y a des navigateurs sur mobile, sur tablette et quelques autres un peu exotiques !

Pour savoir si votre commande fonctionnera, il y a le site <https://www.canluse.com/>

Ce site vous permettra de vérifier une propriété donnée sur votre navigateur et de voir rapidement si ça va marcher...

Par exemple, la propriété *position : fixed* fonctionne sur beaucoup de navigateurs, mais pas tous ! Il y a encore quelques cases rouges. En revanche, *position : sticky* fonctionne beaucoup moins bien !



D'autre part, parfois le navigateur comprend mais n'interprète pas de la façon que vous attendez... Le résultat ne sera donc pas toujours le même d'un navigateur à l'autre. Il faudra souvent faire de nombreux tests sur plusieurs navigateurs et plusieurs tailles d'écran avant de valider un site.

Où écrire le CSS ?

Il y a trois endroits possibles... Un seul est réellement satisfaisant et efficace, mais il peut être utile de connaître les autres ! Prenons un exemple simple, on veut mettre les paragraphes de notre site en bleu... La balise paragraphe est `<p>` la propriété CSS pour mettre le texte en bleu sera `color : blue`

Le plus mauvais : dans la balise elle-même

Si on écrit :

```
<p style="color: blue;">
  Bonjour et bienvenue sur mon site !
</p>
```

Ca va marcher. Mais si on veut afficher tous les paragraphes de la page en bleu, il faut ajouter ce code dans toutes les balises `<p>`. Et si un jour vous ne voulez plus de ce bleu, mais du vert ou du violet, il faudra tout changer... Pas pratique !

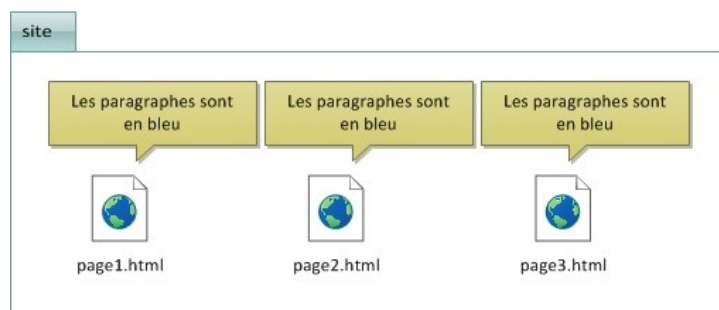
Une solution à ça : dans la section <head>

Si, dans la section `<head>` de votre fichier HTML, vous écrivez :

```
<head>
  <meta charset="utf-8" />
  <title>Premiers tests du CSS</title>

  <style>
    p {
      color: blue;
    }
  </style>
</head>
```

Ça va marcher aussi ! Mais si vous avez plusieurs pages, et que vous voulez la même mise en forme sur toutes, il faudra recopier ce code sur chacune des pages...



C'est moins pire mais toujours pas idéal !

La meilleure solution : Dans un fichier séparé

Le texte dans un fichier, la mise en forme dans un autre. On ne mélange pas, et il suffira de dire au fichier HTML quel fichier CSS utiliser pour que ce soit pris en compte. Ainsi, si vous modifiez le fichier CSS, toutes les pages qui y accèdent seront automatiquement impactées et ça se passera bien avec un minimum de code.

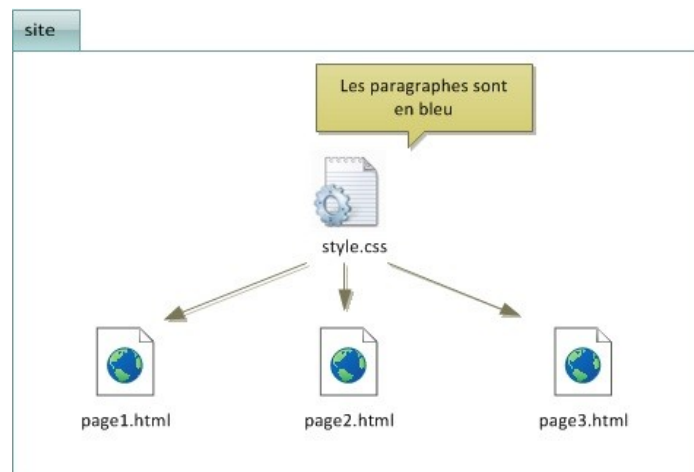
Pour cela, créez un nouveau fichier et écrivez le code suivant dedans :

```
p {  
  color: blue;  
}
```

Enregistrez ce fichier avec l'extension `.css` (par exemple : `style.css`) dans le même dossier que les autres et dans la section `<head>` de votre fichier HTML, ajoutez :

```
<link rel="stylesheet" href="style.css" />
```

Cette simple ligne indique au navigateur d'aller lire le fichier `style.css` qui est une feuille de style (stylesheet) pour savoir comment traiter les balises et comment les afficher.



Si vous modifiez le fichier `style.css`, les trois pages prendront les modifications en compte.

Comment ça s'écrit ?

Je vous ai donné du code CSS sans autre explication... Voyons cela plus précisément

```
p {  
  color: blue;  
}
```

- **p** est le nom de la balise que vous souhaitez modifier
- **color** est la propriété que vous voulez modifier
- **blue** est la valeur à donner à la propriété

Ce sera toujours comme ça ! un fichier CSS aura donc schématiquement la forme suivante :

```
balise1 {
  propriete1: valeur1;
  propriete2: valeur2;
  propriete3: valeur3;
}

balise2 {
  propriete1: valeur1;
  propriete2: valeur2;
  propriete3: valeur3;
  propriete4: valeur4;
}
```

Par exemple, si vous voulez mettre tous les titres principaux en rouge, vous écrirez :

```
h1 {
  color : red;
}
```

Imaginons maintenant que tous les mots mis en avant doivent être affichés en vert. Les balises ** et ** sont concernées. Il faudrait donc écrire :

```
strong {
  color : red;
}

em {
  color : red;
}
```

C'est un peu répétitif, on peut faire mieux très simplement... Deux balises peuvent être séparées par des virgules afin d'afficher la même chose pour les deux.

```
strong, em {
  color : red;
}
```

Des commentaires

Ici aussi, comme dans tous les langages, on peut mettre des commentaires. Comme toujours, ils ne seront pas affichés, ils servent simplement à indiquer des informations pour vous, par exemple pour vous y retrouver dans un looong fichier CSS.

Dans le cas du CSS, il suffit d'encadrer le texte avec les caractères */** au début et **/* à la fin. Par exemple

```
/* écrit le texte en rouge pour ces deux balises */
strong, em {
  color : red;
}
```

Il peut y avoir plusieurs lignes de commentaire. Tant que `*/` n'est pas trouvé, c'est du commentaire !

```
/*
  style.css
  -----
  Cours du lycée St Charles-Ste Croix
*/
```

Mais il y a un mais...

Ce que je vous ai montré jusqu'ici a quand même un défaut : cela implique que TOUS les paragraphes possèdent la même présentation (ici, ils seront donc tous écrits en bleu). Comment faire pour que certains paragraphes seulement soient écrits d'une manière différente ?

On pourrait placer le code CSS dans un attribut `style` sur la balise que l'on vise (c'est la technique que je vous ai présentée un peu plus tôt) mais, comme je vous l'ai dit, ce n'est pas recommandé (il vaut mieux utiliser un fichier CSS externe).

Pour résoudre le problème, on peut utiliser des attributs spéciaux qui fonctionnent sur toutes les balises, ce sont des attributs universels et nous en avons d'ailleurs déjà vu un lors des liens :

L'attribut class et l'attribut id

Les attributs *class* et *id* sont quasiment identiques. Nous avons déjà vu *id* au moment des liens, il permet de définir un identifiant unique pour une balise, nous allons y revenir.

L'attribut class

L'attribut *class* permet lui aussi de donner un nom qui servira à identifier une balise, mais cette fois, plusieurs balises peuvent avoir le même nom. Et une balise peut plusieurs noms. Pour cela, il suffira de séparer les noms par des espaces.

```
<h1 class="titre-principal main"> ... </h1>
<p class="introduction rouge"> ... </p>

```

Par exemple, pour mettre en bleu les coordonnées de votre CV, il faudra d'abord identifier le paragraphe :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="style.css" />
    <title>mon CV</title>
  </head>
  <body>
    <h1>Mon CV</h1>

    <h2>Qui suis-je?</h2>

    <p class="coordonnées">
      Gabriel Gélina<br/>
```

```
12, Rue du chat qui pêche<br/>
72000 Le Mans
</p>
<h2> Mon parcours</h2>
<!-- toute la suite avec d'autres paragraphes... -->

</body>
</html>
```

Maintenant que c'est fait, votre paragraphe est identifié. Il a un nom : *coordonnées*. Vous allez pouvoir réutiliser ce nom dans le fichier CSS pour dire : « Je veux que seules les balises qui ont comme nom **coordonnées** soient affichées en bleu ».

Pour faire cela en CSS, indiquez le nom de votre classe en commençant par un point, comme ci-dessous :

```
.coordonnées {
  color: blue;
}
```

Si plusieurs balises ont la classe **coordonnées**, elles apparaîtront toutes en bleu... Attention, de ce fait, au nom que vous donnez à vos classes... Essayez de trouver des noms significatifs ! On peut très bien donner la classe **coordonnées** à une liste d'expériences pour la mettre en bleu. Ça marchera, mais cela n'a aucun sens !!!

L'attribut id

Lui, il fonctionne exactement de la même manière que *class*, à un détail près : il ne peut être utilisé qu'une fois dans la page. Nous avons déjà vu l'attribut *id* dans le chapitre sur les liens (pour réaliser des liens au sein d'une même page). En pratique, nous ne mettrons des *id* que sur des éléments qui sont uniques dans la page, comme par exemple le logo :

```

```

Si vous utilisez des *id*, lorsque vous définirez leurs propriétés dans le fichier CSS, il faudra faire précéder le nom de l'*id* par un dièse (#) :

```
#logo {
  /* Indiquez les propriétés CSS ici */
}
```

En résumé

Si vous vous mélangez entre *class* et *id* retenez que deux balises peuvent avoir le même nom avec l'attribut *class*. Un nom d'*id* doit, en revanche, être unique dans toute la page HTML.

Des balises universelles...

Il arrivera parfois que vous ayez besoin d'appliquer une *class* (ou un *id*) à certains mots ou à un groupe de balises qui, à l'origine, ne sont pas entourés par des balises.

En effet, le problème de *class*, c'est qu'il s'agit d'un attribut. Vous ne pouvez donc en mettre que sur une balise. Si, par exemple,

je veux modifier uniquement « bienvenue » dans le paragraphe suivant :

```
<p>Bonjour et bienvenue sur mon site !</p>
```

Cela serait facile à faire s'il y avait une balise autour de « bienvenue » mais, malheureusement il n'y en a pas. Par chance, on a inventé... la balise-qui-ne-sert-à-rien.

En fait, on a inventé deux balises dites universelles, qui n'ont aucune signification particulière (elles n'indiquent pas que le mot est important, par exemple). Il y a une différence minime (mais significative !) entre ces deux balises :

- ** ** : c'est une balise de type **inline**, c'est-à-dire une balise que l'on place au sein d'un paragraphe de texte, pour sélectionner certains mots uniquement. Les balises **** et **** sont de la même famille. Cette balise s'utilise donc au milieu d'un paragraphe et c'est celle dont nous allons nous servir pour colorer « bienvenue ».
- **<div> </div>** : c'est une balise de type **block**, qui entoure un bloc de texte. Les balises **<p>**, **<h1>**, etc. sont de la même famille. Ces balises ont quelque chose en commun : elles créent un nouveau « bloc » dans la page et provoquent donc obligatoirement un retour à la ligne. **<div>** est une balise fréquemment utilisée dans la construction d'un design, comme nous le verrons plus tard.

Si on veut mettre juste le mot « bienvenue » en bleu, on pourra donc, par exemple, écrire :

```
<p>
  Bonjour et <span class="bleu">bienvenue</span> sur mon site !
</p>
```

Il suffira d'ajouter le CSS pour la classe *bleu*, et on obtiendra le résultat attendu !

```
.bleu {
  color: blue;
}
```

Notons au passage que si, comme dans l'exemple, on veut le *span* dont la classe est *bleu*, on écrira :

```
span.bleu { /*sans espace */
  color: blue;
}
```

Les sélecteurs avancés

En CSS, le plus difficile est de savoir cibler le texte dont on veut changer la forme. Pour cibler (on dit « sélectionner ») les éléments de la page à modifier, on utilise ce qu'on appelle des **sélecteurs**. Vous en avez déjà utilisé quelques-uns un peu plus tôt dans ce chapitre, résumons-les pour commencer.

Les sélecteurs, que nous avons vus précédemment, sont de loin les plus couramment utilisés. Il faut les connaître par cœur.

Commençons par la base de la base :

```
p {
}
```


... signifie « Je veux toucher tous les paragraphes ». Après, c'est à vous de dire ce que vous faites à ces paragraphes (vous les écrivez en bleu, par exemple).

Nous avons aussi vu :

```
h1, em {  
}
```

... qui signifie « Tous les titres principaux et tous les textes importants ». Nous avons sélectionné deux balises d'un coup.

Et enfin, nous avons vu comment sélectionner des balises précises à qui nous avons donné un nom grâce aux attributs *class* et *id* :

```
.class {  
}  
  
#id {  
}
```

Il existe des dizaines d'autres façons de cibler des balises en CSS ! Nous n'allons pas toutes les voir car il y en a beaucoup et certaines sont complexes, mais voici déjà de quoi vous permettre d'être plus efficaces en CSS !

Les sélecteurs avancés

* : sélecteur universel

```
* {  
}
```

Sélectionne toutes les balises sans exception. On l'appelle le sélecteur universel. Rarement utilisé, mais il est bon d'en avoir entendu parler !

S1 S2: une balise contenue dans une autre

```
h3 em {  
}
```

Sélectionne toutes les balises `<em>` situées à l'intérieur d'une balise `<h3>`. Notez qu'il n'y a pas de virgule entre les deux noms de balises. Ce n'est pas limité à deux... Le code HTML correspondant pourrait être :

`<h3>Titre avec <em>texte important</em></h3>`

S1 + S2 : une balise qui en suit une autre

```
h3 + p {  
}
```

Sélectionne la première balise `<p>` située après un titre `<h3>`. Par exemple :

```
<h3>Titre</h3>  
<p>Paragraphe</p>
```

S[attribut] : une balise qui possède un attribut

```
a[title] {  
}
```

Sélectionne, par exemple, tous les liens <a> qui possèdent un attribut title.

```
<a href="http://site.com" title="Infobulle">
```

S[attribut="Valeur"] : une balise, un attribut et une valeur exacte

```
a[title="Cliquez ici"] {  
}
```

Idem, mais l'attribut doit en plus avoir exactement pour valeur « Cliquez ici ».

```
<a href="http://site.com" title="Cliquez ici">
```

S[attribut*="Valeur"] : une balise, un attribut et une valeur

```
a[title*="ici"] {  
}
```

Idem, l'attribut doit cette fois contenir dans sa valeur le mot « ici » (peu importe sa position).

```
<a href="http://site.com" title="Quelque part par ici">
```

D'autres sélecteurs existent !

Je ne vous ai présenté ici qu'une partie des sélecteurs CSS mais sachez qu'il en existe beaucoup d'autres. Si vous voulez une liste complète, vous pouvez vous renseigner directement à la source : sur le site du W3C !

https://www.w3schools.com/cssref/css_selectors.php

C'est très complet. Certains sélecteurs commencent par les deux-points (:hover par exemple). Ce sont des sélecteurs très particuliers nous y reviendrons un peu plus tard.

Formater le texte

Voilà, vous savez utiliser les sélecteurs et sélectionner le texte que vous voulez mettre en forme... Maintenant, voyons ce qu'il est possible de faire comme mise en page. Nous avons commencé à voir la couleur, mais ce n'est pas tout, fort heureusement.

La taille

Pour modifier la taille du texte, on utilise la propriété CSS **font-size**. Mais pour ce qui est de la valeur à lui donner, c'est un peu plus compliqué ! Il existe en effet de nombreuses unités possibles, relatives ou absolues...

Vous verrez bientôt que ces tailles sont applicables dans de nombreuses situations, pour définir les marges par exemple, pas seulement pour le texte.

Taille absolue

CSS offre donc différentes unités pour exprimer les dimensions. Certaines proviennent de la typographie, comme le point (**pt**) et le pica (**pc**), d'autres sont connues pour leur usage quotidien, comme le centimètre (**cm**) et le pouce (**in**). Et il y a également une unité "magique" inventée spécifiquement pour CSS: le pixel **px**.

Il n'y a pas de restriction à utiliser telle unité à tel ou tel endroit. Si une propriété accepte une valeur en **px** ('font-size: 16px') elle accepte également une valeur en pouces ou en centimètres et vice-versa.

La relation entre les unités absolues est la suivante: 1in = 2.54cm = 25.4mm = 72pt = 6pc

```
p {
  font-size : 14px ; /* Paragraphes de 14 pixels */
}

h1 {
  font-size : 2in ; /* Titres principaux en 2 pouces */
}
```

Sept mots clés ont aussi été créés pour définir les tailles :

- **xx-small** : minuscule ;
- **x-small** : très petit ;
- **small** : petit ;
- **medium** : moyen ;
- **large** : grand ;
- **x-large** : très grand ;
- **xx-large** : euh... gigantesque.

```
p {
  font-size: small;
}

h1 {
  font-size: large ;
}
```

Mais utiliser une unité absolue est déconseillée... En effet, l'élément apparaîtra de façon identique quel que soit le support et sa dimension... Indépendamment des choix de l'utilisateur, un pouce fera toujours 2,54cm ! Ont donc été inventées les unités relatives.

Taille relative

Il y a plusieurs moyens d'indiquer une valeur relative. Deux mots clés, par exemple :

- smaller
- larger

Ces mots clés permettent d'afficher le texte en plus petit ou plus grand que son parent. Ils sont intéressants, mais limités, il n'y en a que 2... Si on veut une taille intermédiaire, ou beaucoup plus grande, c'est impossible... Dans les tailles absolue, nous avons vu des unités comme **px** ou **in**... Il est aussi possible d'utiliser des unités pour définir des tailles relatives... La plus utilisée est sans doute **em**. Il s'agit d'une taille

relative au parent de l'élément ciblé... On indique une valeur par laquelle multiplier la taille de l'élément parent. Ceci peut produire des effets étranges...

Par exemple :

```
<span>Texte <span>plus gros</span> que ça</span>
```

avec

```
span {  
  font-size : 1.5em;  
}
```

produira l'effet suivant :

texte **plus gros** que ça

Pourquoi... Le premier *span* est affiché avec une taille de 1.5 fois la taille de son parent (body, par exemple). Le second *span* est aussi affiché avec une taille de 1.5 fois la taille de son parent... Mais son parent, n'est plus body, c'est le premier *span*... La taille du second *span* est donc de $1.5 * 1.5$ (soit 2.25) fois la taille de body ! Pour éviter cela, quand ça pose problème (parfois c'est un effet voulu), on a inventé l'unité **rem**. Qui sera non plus relative à la taille du parent, mais à la taille de la balise *html*.

Ainsi, si on reprend l'exemple précédent :

```
<span>Texte <span>pas plus gros</span> que ça</span>
```

avec

```
span {  
  font-size : 1.5rem;  
}
```

produira l'effet suivant :

texte pas plus gros que ça

La balise *html* a une taille de caractère, toutes les balises *span* auront une taille 1.5 fois plus grande que celle là.

Il y a beaucoup d'autres unités possibles. Cherchez **font-size** sur votre moteur de recherche favori, vous les trouverez toutes ! De nouvelles unités ont toutefois fait son apparition avec css3, il s'agit des dimensions relatives au support sur lequel c'est affiché. Ainsi, sur un téléphone, une tablette, ou un écran 27 pouces, ça sera proportionnel ! Ce sont, les **unités vh et vw** qui dépendent respectivement de la hauteur et de la largeur de la fenêtre de navigation. Si la taille de la fenêtre du navigateur change, la taille des caractères change aussi !

```
p {  
  font-size : 10vw;  
}
```

La police

C'est un point sensible... Le navigateur ne sait, a priori, afficher que les polices dont il dispose sur l'ordinateur. La bonne nouvelle, c'est que depuis CSS 3, il est possible de faire télécharger automatiquement une police au navigateur.

Modifier la police utilisée

La propriété CSS qui permet d'indiquer la police à utiliser est **font-family**. Vous devez écrire le nom de la police comme ceci :

```
balise {  
  font-family: police;  
}
```

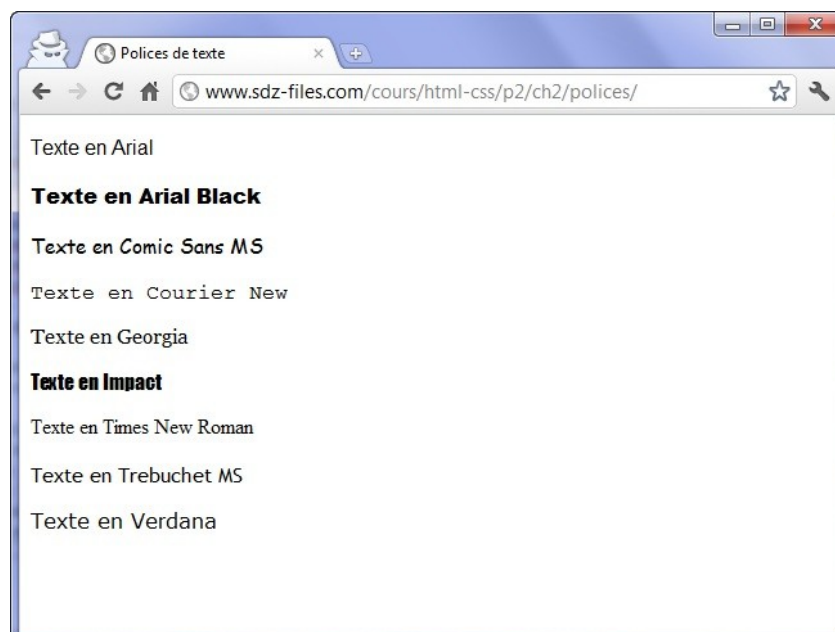
Pour éviter les problèmes si l'internaute n'a pas la même police que vous, on précise en général plusieurs noms de polices, séparés par des virgules :

```
balise {  
  font-family: police1, police2, police3;  
}
```

Le navigateur essaiera d'abord d'utiliser la *police1*. S'il ne l'a pas, il essaiera la *police2*. S'il ne l'a pas, il passera à la *police3*, et ainsi de suite. En général, on indique en tout dernier **serif** ou **sans-serif**, suivant le choix graphique (la différence entre les deux est la présence de petites pattes de liaison en bas des lettres), ce qui correspond à une police par défaut (qui ne s'applique que si aucune autre police n'a été trouvée).

Polices courantes

Il y a un certain nombres de polices "autorisées" car normalement présentes sur tous les ordinateurs... En voici une petite démonstration pour que vous sachiez à quoi elles ressemblent !



On écrira ainsi par exemple :

```
p {  
  font-family: Impact, "Arial Black", Arial, Verdana, sans-serif;  
}
```

Notez les guillemets autour de *Arial Black* afin d'éviter les problèmes avec l'espace !

@font-face

C'est très bien tout ça, mais si on veut une police spécifique, un peu plus originale, ça n'est donc pas sûr de s'afficher correctement... Puisque la police ne sera pas forcément présente sur l'ordinateur où l'on regarde.

Il y a plusieurs formats de polices(eot, svg...), mais un format très courant est le ttf (TrueType Font). On pourra donc écrire :

```
@font-face {  
  font-family: 'SuperPolice';  
  src: url('SuperPolice.eot') format('eot'),  
       url('SuperPolice.woff') format('woff'),  
       url('SuperPolice.ttf') format('truetype'),  
       url('SuperPolice.svg') format('svg');  
}
```

Ensuite, on pourra utiliser *SuperPolice* dans les directives CSS pour utiliser cette police. L'inconvénient majeur de cette technique est que le navigateur va devoir télécharger la ou les polices indiquées et ça peut prendre du temps !

Il est possible d'indiquer une police avec une url absolue et demander le téléchargement à partir d'un site externe, mais le plus simple reste de télécharger la police et de créer ce lien vers le dossier où sont stockés les fichiers.

Alignement du texte

La propriété [text-align](#) s'utilise pour contrôler l'alignement du texte dans la zone de contenu. Les valeurs acceptées sont les suivantes. Elles fonctionnent à peu près de la même manière que dans un traitement de texte :

- `left`: le texte est aligné à gauche.
- `right`: le texte est aligné à droite.
- `center`: le texte est centré.
- `justify`: étale le texte, en faisant varier les espaces entre les mots afin de donner la même largeur à toutes les lignes du texte.

Autres propriétés de mise en forme

[font-style](#) : utilisé pour appliquer ou enlever le style italique. Les valeurs possibles sont les suivantes (vous ne l'utiliserez que rarement, sauf si vous souhaitez désactiver le style italique pour une raison quelconque) :

- `normal` : fige le texte en police normale (suppression du style italique existant).
- `italic` : met le texte en *version italique de la police* si elle est disponible ; si elle n'existe pas, le style italique sera émulé avec l'option oblique à la place.

- **oblique** : force le texte à utiliser une version simulée de fonte italique, créée en *inclinant la version normale*.

font-weight : définit l'épaisseur du texte. La propriété peut avoir de nombreuses valeurs s'il y a de nombreuses variantes de polices disponibles (comme *-light*, *-normal*, *-bold*, *-extrabold*, *-black*, etc.), mais en réalité, vous les utiliserez rarement en dehors de **normal** et **bold** (gras):

- **normal**, **bold** : gras de la police, **normale** ou **bold**.
- **lighter**, **bolder** : définit l'épaisseur de l'élément courant de sorte qu'il soit un cran moins gras ou plus gras que son parent.
- **100-900** : valeurs numériques du taux de gras ; elles permettent un dosage plus fin que les mots-clés ci-dessus, si nécessaire.

text-transform : Vous permet de définir les transformations de votre police. Les valeurs comprennent :

- **none** : aucune transformation.
- **uppercase** : met tout le texte en majuscules.
- **lowercase** : met tout le texte en minuscules.
- **capitalize** : transforme tous les mots en mettant leur première lettre en majuscules.
- **full-width** : force l'écriture de tous les glyphes dans un carré de largeur fixe, similaire à une police à espacement fixe, permettant leur alignement, par ex. caractères latins avec des glyphes de langue asiatique (comme le chinois, le japonais, le coréen.)

text-decoration : active ou désactive les décorations de texte sur les polices (vous les utiliserez principalement pour désactiver le soulignement par défaut sur les liens lors de leur création). Les valeurs disponibles sont :

- **none**: désactive les décorations du texte déjà présentes.
- **underline**: souligne le texte.
- **overline**: trace une ligne au-dessus du texte.
- **line-through**: barre le texte.

Et d'autres encore...

Il est encore possible de réaliser beaucoup de choses... Mettre des ombres définir des espacements, par exemple... Vous en trouverez une liste complète sur la page :

https://www.w3schools.com/css/css_text.asp

Les flottants

Souvent, on souhaite faire des mises en page de ce genre :



Lorem ipsum dolor sit amet,
consectetur adipiscing elit. Donec
vitae lorem imperdiet lacus molestie
molestie. Cum sociis natoque penatibus
et magnis dis parturient montes, nascetur ridiculus
mus. Donec eu purus. Phasellus metus lorem,
blandit et, posuere quis, tincidunt vitae, ante.
Vivamus consequat mauris a diam. Vivamus nibh
erat, hendrerit nec, aliquet ut, hendrerit quis, nunc.
Vestibulum et turpis et elit tempor euismod.

Jusqu'à présent, c'était impossible ! Eh bien grâce aux flottants, c'est assez simple.

Créer un flottant

Flotter en anglais, ça se dit float, ça sera donc le nom de cette propriété magique ! Elle peut prendre deux valeurs : left et right ! Soit, ça flotte à gauche, soit ça flotte à droite.

Il faut, comme toujours, clairement identifier la balise sur laquelle on veut appliquer la propriété, par exemple, si on attribue la classe *imageflottante* à une balise *img*.

```
<p>  
    
  Ceci est un texte normal de paragraphe, écrit à la suite de l'image et  
  qui l'habillera car l'image est flottante.  
</p>
```

Il suffira d'ajouter la propriété *float* à la classe choisie

```
.imageflottante {  
  float: left;  
}
```

pour que l'habillage fonctionne comme dans l'exemple du haut de la page...

Stopper un flottant

Attention cependant, on peut aussi vouloir que ça s'arrête... Les éléments flottants sortent du flux normal de la page. Cela peut produire des effets bizarres...

Souvent, par exemple, les menus sont composés d'éléments de liste, qui normalement sont affichés l'un en dessous de l'autre. On les fait donc flotter pour les mettre côte à côte. Mais la suite du texte, qui devrait être affiché en dessous, va donc apparaître à côté aussi ! Alors qu'on voudrait qu'il soit en dessous...

Par exemple, avec le html

```
<ul class="menu">  
  <li>menu 1</li>  
  <li>menu 2</li>  
  <li>menu 3</li>  
  <li>menu 4</li>  
</ul>
```



```
<h1>Qu'est-ce que c'est <span>?</span></h1>
```

et le CSS

```
ul.menu li {  
  float: left;  
}
```

On obtiendra le résultat suivant :

• menu1menu2menu3menu 4

Qu'est-ce que c'est ?

Le titre apparaît à la suite du menu alors qu'on le voudrait en dessous !

Pour arrêter un flottant, il existe la propriété *clear* qui met un terme au flottant et qui, donc, renvoie l'élément dans le flux normal, à la ligne.

La propriété *clear* peut prendre trois valeurs : *left*, *right* et *both* (gauche, droite ou les deux) suivant le côté où l'on a fait flotter l'élément... Cela dit, en pratique, le plus souvent, on utilisera *both*... Comme ça, il n'y a pas à réfléchir !

Ici, si on ajoute simplement le CSS :

```
ul.menu + h1 { /* la balise h1 qui suit le menu */  
  clear: both;  
}
```

On obtient bien :

• menu1menu2menu3menu 4

Qu'est-ce que c'est ?

Les couleurs et le fond

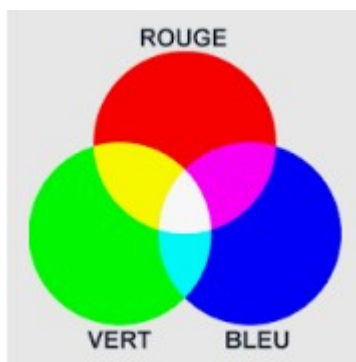
Dans les exercices précédents, vous avez utilisé des couleurs en donnant leur nom. C'est le plus simple, mais cela présente un inconvénient majeur, il n'y en a que 16 ! Suivant les navigateurs, il peut y en avoir d'autres, mais ce n'est pas normalisé, je ne donnerai que les 16 officielles !

white	
silver	
gray	
black	
red	
maroon	
lime	
green	
yellow	
olive	
blue	
navy	
fuchsia	
purple	
aqua	
teal	

Vu que la plupart des écrans sont capables d'afficher 16 millions de couleurs, les créateurs graphiques ne se limitent jamais à cela. Il faut donc d'autres moyens d'indiquer les couleurs, afin d'obtenir exactement ce qui est demandé par les graphistes...

Codage des couleurs

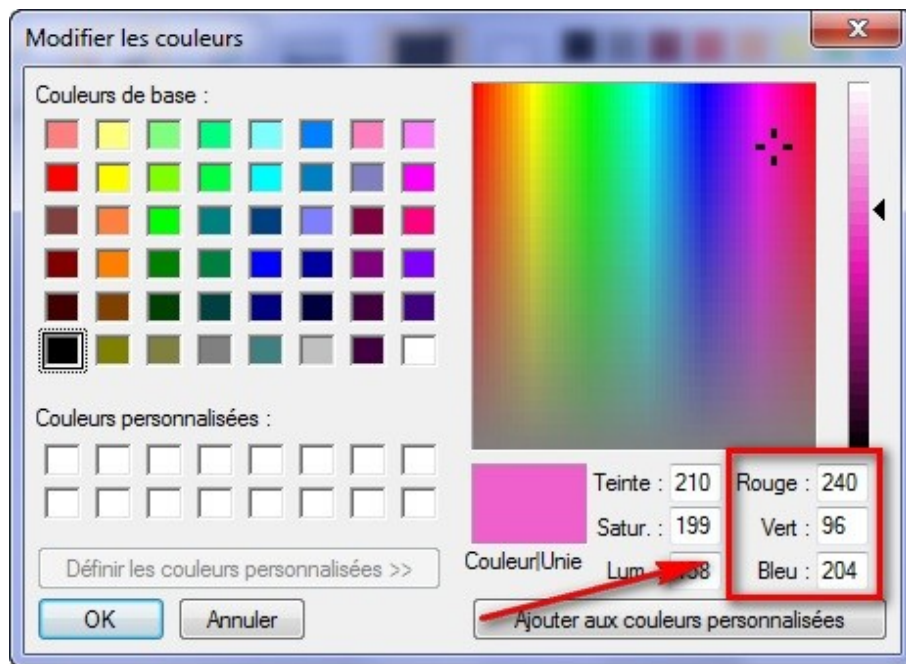
Pour faire simple, dans le cas d'un écran, une couleur est le mélange des trois composantes Rouge, Vert et Bleu. On parlera parfois de codage RVB ou RGB pour Red, Green, Blue en anglais.



Rouge + Vert donne jaune ; Vert + Bleu donne Cyan : Bleu + Rouge donne Magenta. En fonction de la quantité de chaque composante, on obtient tous les dégradés possibles... Notez que si toutes les composantes sont au maximum, on obtient du blanc. En revanche, si elles sont au minimum, on obtient du noir.

Chaque quantité est codée sur un octet (8 bits), elle peut donc prendre une valeur allant de 0 à 255 (256 possibilités pour chaque composante. Ce qui donne bien $256 * 256 * 256 = 16777216$, soit plus de 16 millions).

En utilisant simplement Paint sous Windows, vous pouvez choisir précisément vos couleurs et obtenir la valeur de chaque composante.



Codage en CSS

Nous avons déjà vu comment définir la couleur d'un texte avec la propriété *color*. Jusqu'à présent, pour sa valeur, nous étions limités aux 16 couleurs prédéfinies. Il sera maintenant possible de donner précisément la quantité de chaque composante en utilisant le codage vu au dessus.

Valeur rgb

Il est possible de donner directement en décimal la quantité de chaque composante en utilisant la valeur :

```
color : rgb(240, 96, 204) ;
```

donnera le texte écrit dans le joli rose de l'image précédente.

Valeur hexadécimale

Le plus souvent, on notera la couleur en hexadécimal. Il y a donc trois octets à coder, un pour le rouge, un pour le vert et un pour le bleu.

Ils seront mis dans un seul code hexadécimal précédé du caractère #. Par exemple :

```
color : #FFFFFF ; /* Blanc puisque chaque couleur est au maximum */  
color : #000000 ; /* Noir */  
color : #F060CC ; /* Le même rose que précédemment */
```

Couleur de fond

Nous avons revu que *color* permet de définir la couleur du texte. Il est aussi possible, de la même façon, de définir la couleur de l'arrière-plan. En anglais, arrière-plan se dit *background*, on utilisera donc la propriété *background-color* à laquelle on donnera la couleur voulue. Pour cela, les deux méthodes de codage des couleurs sont bien évidemment possibles. Par exemple, on pourra écrire :

```
h1 { /* pour définir l'arrière-plan des titres principaux */  
  background-color : #C0C0FF ; /* un bleu clair */  
}
```

Pour faire un choix

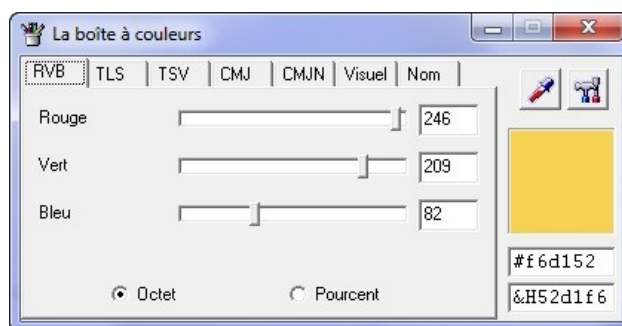
Il existe de nombreuses possibilités, y compris sur internet, pour trouver un code de couleur, par exemple :

<https://htmlcolorcodes.com/fr/>

<https://www.toutes-les-couleurs.com/code-couleur-rvb.php>

Il est aussi possible d'utiliser Paint, voire Photoshop pour trouver une couleur et obtenir ses composantes RGB.

Un petit logiciel Windows peut aussi aider, Il s'appelle "La boîte à couleurs" et est très simple d'utilisation... A vous de choisir, les solutions sont nombreuses.



Généralement, quand on travaille avec un graphiste, ça

fait partie de son travail de vous donner les codes des couleurs.

Images d'arrière-plan

Il est donc simple de définir une couleur en arrière-plan. Ce n'est pas beaucoup plus compliqué d'y mettre une image. Il va cependant falloir faire quelques choix pour définir comment cette image sera affichée...

Tout d'abord, la propriété de base, arrière-plan se dit toujours `background`, ce sera donc *background-image* ! Pour la valeur, on notera `url("nom_de_l_image.png")`. Par exemple :

```
body {  
  background-image: url("neige.png");  
}
```

Options disponibles pour l'image de fond

On peut compléter la propriété `background-image` que nous venons de voir par plusieurs autres propriétés qui permettent de changer le comportement de l'image de fond.

background-attachment : fixer le fond ou pas

La propriété CSS `background-attachment` permet de « fixer » le fond. L'effet obtenu est intéressant car on voit alors le texte « glisser » par-dessus le fond. Deux valeurs sont disponibles :

```
fixed : l'image de fond reste fixe ;  
scroll : l'image de fond défile avec le texte (par défaut).
```

par exemple :

```
body {  
  background-image: url("neige.png");  
  background-attachment: fixed; /* Le fond restera fixe */  
}
```

background-repeat : répétition du fond

Par défaut, l'image de fond est répétée en mosaïque.

Vous pouvez changer cela avec la propriété *background-repeat* :

```
no-repeat : le fond ne sera pas répété. L'image sera donc unique sur la page.  
repeat-x : le fond sera répété uniquement sur la première ligne,  
horizontalement.  
repeat-y : le fond sera répété uniquement sur la première colonne,  
verticalement.  
repeat : le fond sera répété en mosaïque (par défaut).
```

Par exemple :

```
body{  
  background-image: url("soleil.png");  
  background-repeat: no-repeat;  
}
```

background-position : position du fond

On peut indiquer où doit se trouver l'image de fond avec *background-position*. Cette propriété n'est intéressante que si elle est combinée avec *background-repeat: no-repeat*; (un fond qui ne se répète pas).

Vous pouvez donner à *background-position* deux valeurs en pixels pour indiquer la position du fond par rapport au coin supérieur gauche de la page (ou du paragraphe, si vous appliquez le fond à un paragraphe).

Ainsi, si vous tapez :

```
background-position: 30px 50px;
```

... votre fond sera placé à 30 pixels de la gauche et à 50 pixels du haut. Il est aussi possible d'utiliser ces valeurs en anglais : **top** : en haut ; **bottom** : en bas ; **left** : à gauche ; **center** : centré ; **right** : à droite.

Il est possible de combiner ces mots. Par exemple, pour aligner une image en haut à droite, vous taperez :

```
background-position: top right;
```

Ainsi, si je veux afficher un soleil en image de fond (figure suivante), en un unique exemplaire (no-repeat), toujours visible (fixed) et positionné en haut à droite (top right), je vais écrire ceci :

```
body {  
  background-image: url("soleil.png");  
  background-attachment: fixed; /* Le fond restera fixe */  
  background-repeat: no-repeat; /* Le fond ne sera pas répété */  
  background-position: top right; /* Le fond sera placé en haut à droite */  
}
```



Combiner les propriétés

Si vous utilisez beaucoup de propriétés en rapport avec le fond (comme c'est le cas sur ce dernier exemple), vous pouvez utiliser une sorte de « super-propriété » appelée *background* dont la valeur peut combiner plusieurs des propriétés vues précédemment : background-image, background-repeat, background-attachment et background-position.

On peut donc tout simplement écrire :

```
body {  
  background: url("soleil.png") fixed no-repeat top right;  
}
```

C'est la première « super-propriété » que je vous montre, il y en aura d'autres. Il faut savoir que :

L'ordre des valeurs n'a pas d'importance. Vous pouvez combiner les valeurs dans n'importe quel ordre.

Vous n'êtes pas obligés de mettre toutes les valeurs. Ainsi, si vous ne voulez pas écrire *fixed*, vous pouvez l'enlever sans problème.

Plusieurs images de fond

Depuis CSS3, il est possible de donner plusieurs images de fond à un élément. Pour cela, il suffit de séparer les déclarations par une virgule, comme ceci :


```
body {
    background: url("soleil.png") fixed no-repeat top right, url("neige.png")
    fixed;
}
```

La première image de cette liste sera placée par-dessus les autres (figure suivante). Attention donc, l'ordre de déclaration des images a son importance : si vous inversez le soleil et la neige dans le code CSS précédent, vous ne verrez plus le soleil puisque la neige est répétée et occupe donc toute la fenêtre.



La transparence

Le CSS nous permet de jouer très facilement avec les niveaux de transparence des éléments ! Pour cela, nous allons utiliser des fonctionnalités de CSS3 : la propriété *opacity* et la notation *RGBA*.

La propriété *opacity*

La propriété *opacity*, très simple, permet d'indiquer le niveau d'opacité (c'est l'inverse de la transparence). Avec une valeur de 1, l'élément sera totalement opaque : c'est le comportement par défaut. Avec une valeur de 0, l'élément sera totalement transparent.

Il faut donc choisir une valeur comprise entre 0 et 1. Ainsi, avec une valeur de 0.6, votre élément sera opaque à 60%... et on verra donc à travers !

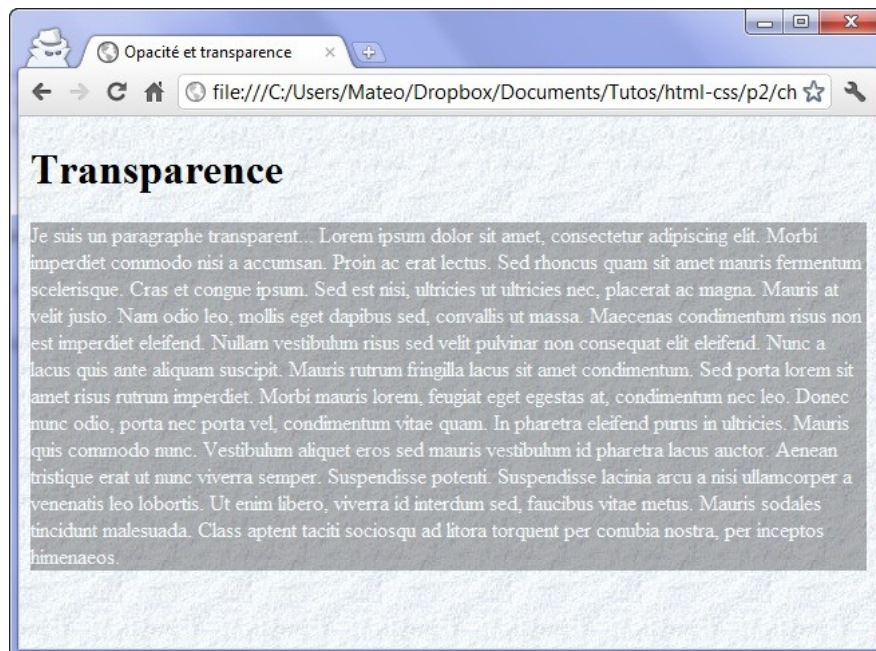
Voici comment on peut l'utiliser :

```
p {
    opacity: 0.6;
}
```

Voici un exemple qui va nous permettre d'apprécier la transparence. Vous en trouverez le rendu à la figure suivante.

```
body {
  background: url('neige.png');
}

p {
  background-color: black;
  color: white;
  opacity: 0.3;
}
```



Si vous appliquez la propriété `opacity` à un élément de la page, tout le contenu de cet élément sera rendu transparent (même les images, les autres blocs à l'intérieur, etc.). Si vous voulez juste rendre la couleur de fond transparente, utilisez plutôt la notation `RGBA` que nous allons découvrir.

La notation `RGBA`

CSS3 nous propose une autre façon de jouer avec la transparence : la notation `RGBA`. Il s'agit en fait de la notation `RGB` que nous avons vue précédemment, mais avec un quatrième paramètre : le niveau de transparence (appelé « canal alpha »). De la même façon que précédemment, avec une valeur de 1, le fond est complètement opaque. Avec une valeur inférieure à 1, il est transparent.

```
p {
  background-color: rgba(255, 0, 0, 0.5); /* Fond rouge à moitié transparent */
}
```

C'est aussi simple que cela. Vous pouvez obtenir exactement le même effet qu'avec `opacity` juste en jouant avec la notation `RGBA`, essayez !

De nombreuses autres propriétés...

Il existe beaucoup de propriétés CSS pour la mise en page de vos textes et images, nous en avons vu quelques unes pour d'autres, souvenez-vous que Google est votre ami !

Voyons maintenant les modèles de boîtes...

Structurez vos pages !

Nous avons vu comment créer des blocs avec la balise <div>. Mais à force de mettre des div et des div dans tous les sens, on finit par ne plus savoir qui fait quoi... HTML5 a apporté quelques balises complémentaires permettant de structurer les pages...

Il n'est pas nécessaire de les utiliser, mais plus vos pages seront complexes, plus il sera pertinent de le faire. Pour cela, comme vous le verrez en exemple, à la fin de ce chapitre, faites un petit schéma avec votre structure avant de commencer quoique ce soit. Ensuite écrivez le HTML et finalement, mettez le tout en forme en écrivant le CSS !

Mais revenons à ces balises structurantes. Elles sont au nombre de 6 essentielles...

<nav>

<nav> permet de définir une partie réservée à la navigation à l'intérieur du site, c'est à dire les menus. Jusqu'à présent, les menus étaient fait avec une liste, ce sera toujours le cas, mais cette liste sera intégrée dans une balise <nav>. Ainsi, on saura immédiatement que cette liste est dédiée aux menus principaux du site.

```
<nav id="menu-principal">
  <ul class="menu">
    <li>menu 1</li>
    <li>menu 2</li>
    <li>menu 3</li>
    <li>menu 4</li>
  </ul>
</nav>
```

Ça n'a l'air de rien, mais quand vous ferez des sites avec des menus complexes, ou en responsive avec des menus "hamburgers" qui s'affichent sur mobile, tout apparaîtra dans des listes et sera intégré dans la partie <nav> réservée à la navigation. Ça vous permettra de vous y retrouver très rapidement.

<header>

C'est l'entête de la page. Souvent on y trouvera une image d'entête ou un logo, parfois un slider de présentation. Un slider est composée d'une série d'images qui défilent pour donner des informations importantes à mettre en avant ou des articles récents... Son utilisation est simple, comme pour toutes les balises, on l'écrit simplement entre chevrons. En général, il y a un seul header...

```
<header>
  contenu de l'entête...
  images, logos ou autres...
</header>
```

<section>

Comme son nom l'indique, il s'agit d'une section d'un site. C'est à dire une partie clairement définie. Il peut y avoir autant de sections que vous voulez, ça ne pose pas de problème.

```
<section>
```

```
    contenu de section...
    articles ou toute autre balise vue jusqu'à présent...
</section>
```

Une section pourra aussi contenir un header et un footer spécifiques à cette section !

<article>

Cette balise permet de faire à peu près la même chose que la précédente, mais en général vous trouverez des articles dans une section. Un article fait référence à une information simple, un article de blog, par exemple. Dans chaque article, il peut y avoir des textes et des images. Il est également possible, bien entendu, de mettre des titres, des div et toutes les balises que vous connaissez à l'intérieur. L'objectif est vraiment de structurer les pages pour vous y retrouver.

On peut imaginer, la structure suivante, par exemple :

```
<section id="articles-recents">
  <article>
    premier article...
  </article>
  <article>
    second article...
  </article>
  <article>
    troisième article...
  </article>
</section>
```

<footer>

C'est la pied de page. Le plus souvent, on y trouvera des informations relatives au copyright, conditions générales ou d'autres informations un peu secondaires. Il peut arriver, sur certains sites que le footer contienne un menu reprenant certaines tâches du menu principal. Dans ce cas, il pourra contenir une balise <nav> !

```
<footer>
  contenu du pied de page...
  Copyright, conditions générales ou autres...
</footer>
```

<aside>

Cette balise n'est pas toujours utilisée, mais elle peut être utile aussi ! Dans les pages, il y a le contenu principal avec le header, le footer, les sections, les articles, etc.

La balise aside sert simplement à indiquer qu'un certain contenu n'est pas directement en rapport avec le contenu principal. Une sorte d'aparté où l'on pourra mettre des informations moins importantes... Comme pour les autres, et comme toujours en HTML, elle s'écrit comme ceci :

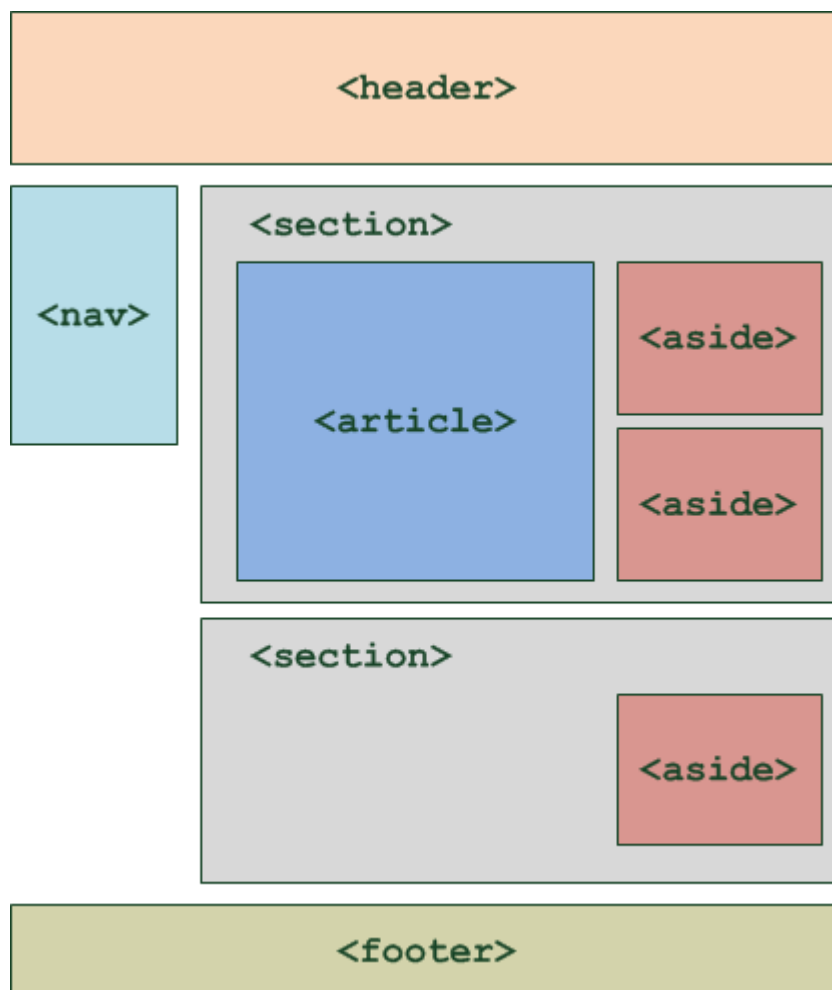
```
<aside>
  contenu en aparté...
</aside>
```

Bonus la balise <main>

C'est la balise qui contient la partie principale du site. Pour ma part, je pense que cette balise est rarement utile ! Elle rejoint en de nombreux points la balise <body> qui, je vous le rappelle contient l'intégralité de la page. Certains site ont une partie un peu secondaire, dans ce cas, pourquoi pas, mais c'est rare. A mon sens, il est préférable de ne pas l'utiliser plutôt que de mal l'utiliser !

Par exemple

Je vous disais au début qu'il est parfois bon de faire un petit schéma pour définir ce qu'on veut. Cela peut par exemple, ressembler à ça :



Attention, ceci n'est qu'un exemple... Rien ne vous empêche de mettre votre menu en haut ou à droite, de mettre plusieurs articles dans la section, etc.

Enfin, une section ne contient pas forcément un ou des articles <article> et des <aside> . Utilisez ces balises uniquement si vous en avez besoin. Rien ne vous interdit de créer des sections contenant seulement des paragraphes, par exemple.

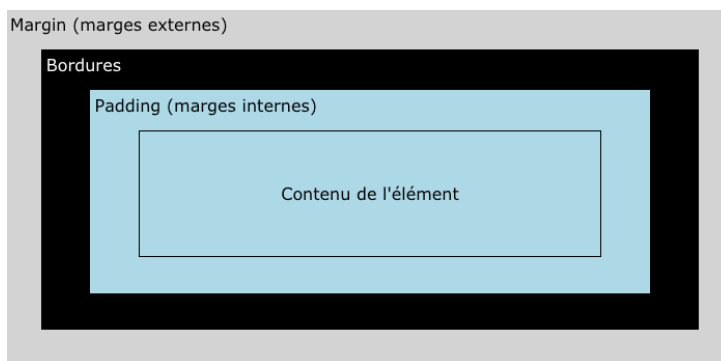
Ces balises sont uniquement destinées à vous aider à vous retrouver dans les pages complexes, elles n'auront aucun impact direct sur la mise en page. Toute la partie relative à la mise en page sera faite en CSS avec les propriétés que nous avons vues ou que nous allons voir.

Boîtes et modèles de boîtes

Vous avez peut-être approché les propriétés **padding** et **margin**, revenons sur ce concept et quelques autres...

Tout d'abord, je vous rappelle les deux types de balises que nous avons approchés : les balises inline et les balises de type bloc. Sauf indication contraire, une balise de type bloc "implique" un retour à la ligne quand elle est fermée... Par exemple, la balise **div**, les balises **h1** à **h6** ou encore la balise **p** sont des blocs, tout comme toutes les balises structurantes. Les balises **strong**, **em** ou même **img** sont, en revanche, des balises inline. Il n'y a pas de retour à la ligne, toujours sauf indication contraire.

Les boîtes, dans le principe c'est simple. Une balise de type bloc est une boîte. Sur une boîte, il y a plusieurs propriétés intéressantes. Voyons d'abord un schéma :



- La première boîte, centrale, va être composée du contenu de l'élément en lui-même ;
- La deuxième boîte va être composée de la première boîte ainsi que des marges internes de l'élément ;
- La troisième boîte va être composée de la deuxième boîte et des bordures de l'élément ;
- La quatrième boîte va être composée de la troisième boîte et des marges externes de l'élément.

Notez au passage les couleurs. La deuxième boîte est toute de la même couleur. C'est parce que si vous affectez une couleur de fond au bloc. Toute cette boîte prendra la couleur, pas seulement la première boîte. Autrement dit, tout ce qui est à l'intérieur de la bordure.

Les propriétés **width** et **height**

Ces propriétés vont servir à déterminer les dimensions d'une boîte. **width** pour la largeur et **height** pour la hauteur mais ce n'est pas tout à fait aussi simple... De quelle boîte parle-t-on ? La première, deuxième, troisième ou quatrième boîte du schéma ?

La propriété **box-sizing** va nous permettre d'indiquer que l'on souhaite inclure les marges internes et les bordures dans le calcul de la taille d'un élément.

Nous allons pouvoir fournir l'un des mots clefs suivants à cette propriété :

- **content-box** : Les dimensions définies pour l'élément vont s'appliquer à sa boîte de contenu. Donc la première boîte du schéma. Toute marge interne ou bordure ajoutées ensuite vont augmenter la taille de l'élément ; c'est la valeur par défaut.
- **border-box** : les dimensions définies pour l'élément vont s'appliquer à la boîte de contenu + le padding + les bordures. En ajoutant ou en augmentant la taille des marges internes ou des bordures, la taille de l'élément ne change pas mais son contenu sera compressé.

Notez que les marges externes d'un bloc ne seront jamais prises en compte dans ses dimensions.

Maintenant, soit le code HTML suivant :

```
<h1>Un titre</h1>

<p class="p1">Un premier paragraphe</p>
<p class="p2">Un autre paragraphe</p>

<span class="s1">Un élément span</span>
<span class="s2">Un autre span</span>
```

associé au code CSS suivant :

```
.p1{
  background-color: lightBlue;
  width: 400px;
  height: 100px;
}

.p2{
  background-color: lightGreen;
}

.s1{
  background-color: orange;
  width: 400px;
  height: 100px;
}

.s2{
  background-color: yellow;
}
```

Cela va produire le résultat suivant :



Déjà, les propriétés `width` et `height` n'ont d'effet que sur les balises de type bloc, les `span` (qui sont de type inline) ne sont pas affectées.

Pour `p1` (le bloc bleu clair) on lui donne une hauteur et une largeur, elles sont appliquées. Pour le bloc `p2` (le bloc vert), en revanche, on ne donne rien. Dans ce cas, la largeur sera à 100% de son conteneur et la hauteur sera celle de son contenu... Ici, une ligne de texte.

Problèmes de dépassement

Il peut arriver que la taille d'un bloc dépasse la taille de son parent ! Par exemple avec les codes suivants :

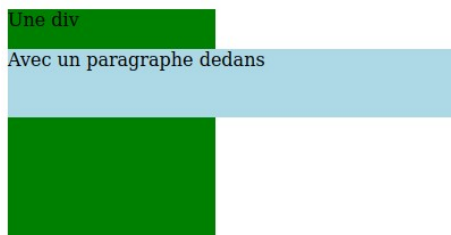
```
<div>
  Une div
  <p>Avec un paragraphe dedans</p>
</div>
```

et pour le CSS :

```
div {
  background-color: green;
  height: 200px;
  width: 10%;
}

p {
  background-color: lightBlue;
  width: 400px;
  height: 60px
}
```

produira le résultat suivant :



Le paragraphe bleu sort de la div verte ! Pour éviter cela une propriété CSS existe. Il s'agit de **overflow**. On peut lui dire quoi faire si ce cas se produit. Plusieurs valeurs sont possibles.

visible	La partie qui dépasse est visible.
hidden	La partie qui dépasse est masquée.
scroll	Une barre de défilement apparaît sur le parent.

Notez qu'il est possible de définir un comportement en horizontal et vertical avec **overflow-x** et **overflow-y**.

La propriété padding

Comme nous l'avons vu sur le premier schéma, les marges intérieures se trouvent entre le contenu de l'élément et sa bordure. Ainsi, définir une marge intérieure importante va éloigner la bordure de l'élément de son contenu. Pour rappel, si on définit une couleur de fond pour notre élément, celle-ci s'applique également dans l'espace correspondant aux marges intérieures. Généralement les unités utilisées seront **px** ou **em**, mais on peut aussi utiliser le **%** qui définira une marge comme pourcentage de son parent.

Une boîte a quatre côtés et il va être possible de définir une marge différente sur chacun d'eux. Tout d'abord, le plus simple, pour définir la même marge sur tout les côtés, on écrira simplement :

```
padding : 25px;
```

Si on souhaite une marge différente en fonction du côté, vous pourrez utiliser les propriétés **padding-top**, **padding-right**, **padding-bottom** et **padding-left** (respectivement pour le haut, la droite, le bas et la gauche)

Il sera également possible de définir les différents côtés dans la propriétés padding elle-même ! Ainsi

```
padding : 10px 20px 30px 40px;
```

définira 10px en haut, 20px à droite, 30px en bas et 40px à gauche. Ou encore :

```
padding : 10px 20px;
```

définira 10px en haut et en bas et 20px à droite et à gauche.

Notez tout de même que le **padding** a un impact sur les éléments, et ceci qu'ils soient de type **block** ou **inline**. On peut donc définir un padding sur un span, cela dit, ce padding sur les éléments inline n'aura pas d'impact sur les autres éléments autour de lui... Cela peut être utile pour définir un cadre de couleur un peu plus large autour d'un mot...

La propriété margin

Nous avons vu que les marges intérieures se trouvent entre le contenu de l'élément et sa bordure. Les marges extérieures, au contraire, vont définir la taille de l'espace autour d'un élément.

- Des valeurs de type longueur, généralement en **px** ou en **em** ;
- Des valeurs de type pourcentage. Dans ce cas, le % indiqué est calculé par rapport à la taille de l'élément parent ;
- Le mot clé **auto**. Ce mot clé va surtout nous servir pour centrer des blocs dans leur parent.

Le principe de nommage des propriétés est exactement le même que pour **padding** avec **margin-top**, **margin-right**, **margin-bottom** et **margin-left** ; Ou la superpropriété **margin** avec ses différentes possibilités pour indiquer les valeurs.

Quelques particularités sont cependant à signaler...

Fusion des marges

Si un bloc a une marge basse de 20px et que le bloc en dessous a une marge haute de 100px, l'espace entre les deux ne sera pas de 120px, mais les marges seront fusionnées et il ne restera au final que la plus grande des deux... ici 100px.

De la même manière, il va y avoir fusion entre un élément parent et son premier enfant s'il n'y a aucune bordure ou remplissage entre sa marge haute et la marge haute de son enfant. idem pour la marge basse du dernier enfant.

Cela peut éviter des surprises de le savoir !...

Centrer un élément

La propriété **margin** va souvent être utilisée pour centrer horizontalement un élément dans son parent.

Pour faire cela, nous allons devoir définir des marges externes **auto** à gauche et à droite de l'élément qui devra être centré.

Attention pour le centrage vertical, c'est une toute autre affaire !!! Sauf pour du texte, où il suffit de définir la hauteur de ligne avec **line-height** pour voir le texte centré verticalement... Si vous voulez centrer une div

verticalement dans une autre, il faudra faire autrement, mais rien n'est impossible... ou presque ! Nous y viendrons avec les flexbox, un peu plus tard.

La propriété border

Les bordures vont être définies par trois caractéristiques en CSS : une épaisseur (ou largeur), un style et une couleur.

- La propriété `border-width` va nous permettre de définir la largeur (ou « l'épaisseur ») d'une bordure ;
- La propriété `border-style` va nous permettre de définir le style d'une bordure ;
- La propriété `border-color` va nous permettre de définir la couleur d'une bordure.
- La superpropriété `border` permettra de définir les 3 d'un coup !

border-width : l'épaisseur

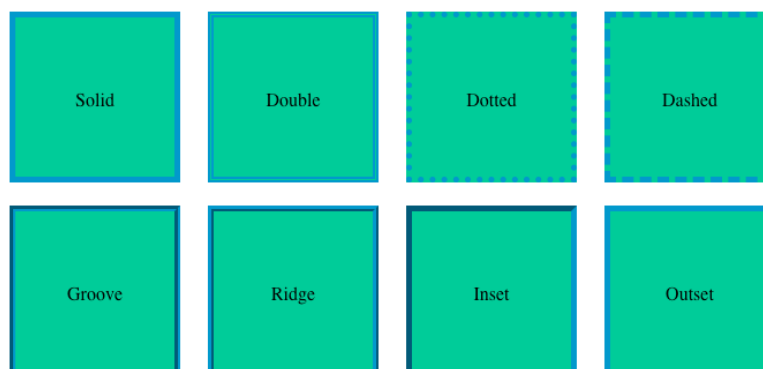
Elle peut être définie avec le mot clé **thin**, **medium** ou **thick**, mais le plus souvent, on donnera directement une épaisseur en **px** ou en **em**.

border-style : l'apparence

Plusieurs mots clés sont utilisables comme valeurs pour définir le style d'une bordure :

Valeur	Description
solid	Bordure solide simple (un trait)
double	Bordure solide double
dotted	Bordure en pointillés
dashed	Bordure constituée de tirets
groove	Bordure incrustée avec effet 3D. L'effet produit est l'inverse de <code>ridge</code>
ridge	Bordure en relief avec effet 3D. L'effet produit est l'inverse de <code>groove</code>
inset	La bordure donne l'effet que la boîte représentant l'élément est enfoncée dans la page. L'effet produit est l'inverse de <code>outset</code>
outset	La bordure donne l'effet que la boîte représentant l'élément est en relief par rapport au reste de la page. L'effet produit est l'inverse de <code>inset</code>

Et voilà à quoi ça ressemble :



border-color : la couleur

Comme toujours une couleur pourra être définie en utilisant un mot clé prédéfini (red, green ou encore lightblue), ou en indiquant une valeur hexadécimale, par exemple: #00CC99 correspond au vert des exemples ci-dessus... Il sera également possible de définir la couleur avec les "commandes" rgb() ou rgba() contenant la quantité de rouge, vert et bleu plus la transparence pour le rgba.

border : superpropriété pour tout définir d'un coup

On pourra par exemple écrire en CSS :

```
border: 3px ridge #00CC99;
```

pour définir l'épaisseur le style et la couleur dans la même commande. Notez que l'ordre n'a pas d'importance...

Chaque côté différent

On ne veut pas toujours de bordure partout autour, par exemple... Aussi, comme pour les autres, il sera possible de définir indépendamment chaque côté... Par exemple :

```
border-top : 5px double green;
```

définira la bordure du haut. **border-right**, **border-bottom** et **border-left** existent aussi ainsi que toutes les variations **border-right-style**, **border-left-width** et ainsi de suite, vous avez compris le principe !

Coins arrondis

Le CSS3 relatif aux bordures a apporté de nouvelles fonctionnalités nous permettant de personnaliser encore davantage nos bordures. Parmi celles-ci, l'une des nouveautés les plus attendues était la possibilité de créer des bordures arrondies.

Nous allons donc pouvoir créer des bordures arrondies en CSS (**ou plus exactement arrondir les bords d'un élément HTML**) en utilisant la propriété **border-radius**.

Notez que l'arrondi créé va s'appliquer aux bords de l'élément en soi. Cela signifie que nous n'avons pas forcément besoin de définir une quelconque bordure avec **border** pour que **border-radius** s'applique.

La propriété **border-radius** va prendre deux valeurs représentant la dimension du « rayon » sur l'axe des X (axe horizontal) et la dimension du « rayon » sur l'axe des Y (axe vertical) d'une ellipse qui va servir à définir la forme de l'arrondi.

Ces deux valeurs dont être séparées par un slash comme ceci : **border-radius: X/Y**. La deuxième valeur est facultative et si elle est omise elle sera considérée comme égale à la première par défaut.

Par exemple avec le HTML :

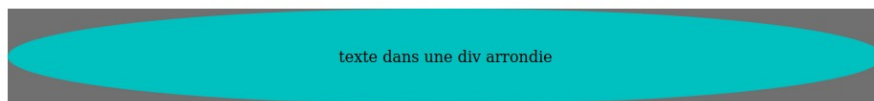
```
<div>
  <p>texte dans une div ovale</p>
</div>
```

et le CSS

```
div{
  width: 50%;
  background-color: #707070; /*Fond gris*/
}

p{
  text-align:center; /*Texte centré horizontalement*/
  line-height: 100px; /*Texte centré verticalement*/
  background-color: #00C0C0; /*Fond bleu vert*/
  border-radius: 50%;
}
```

aura le résultat suivant :



En effet, on demande un arrondi sur la moitié de la largeur horizontalement ET verticalement... La div étant rectangulaire, ça fait un ovale.

Cela peut aussi, par exemple, être utilisé pour afficher des images arrondies, ou plus simplement avec les coins arrondis !...



Et des ombres portées

Finalement, nous pouvons également définir une ombre à nos boîtes ! pour cela 6 valeurs peuvent être fournies à la propriété `box-shadow`. Seules deux sont obligatoires, les autres sont éventuellement indiquées dans l'ordre.

1. **L'inset** : si on précise le mot clef `inset`, alors l'ombre sera tournée vers l'intérieur de la boîte c'est à dire interne à l'élément. Si on omet la valeur (cas par défaut), alors l'ombre sera externe ;
2. **Le décalage horizontal de l'ombre** (obligatoire). Cette valeur est une longueur qu'on va donc pouvoir préciser en unités absolues (px par exemple) ou relatives (em par exemple ; attention les pourcentages ne sont pas acceptés). Si la valeur passée est positive, l'ombre sera projetée sur une distance égale à la valeur passée vers la droite en partant de l'extrémité droite de la boîte. Dans le cas contraire, l'ombre sera projetée à gauche de la boîte en partant de son extrémité gauche ;
3. **Le décalage vertical de l'ombre** (obligatoire). Cette valeur est également une longueur et va pouvoir prendre les mêmes unités que le décalage horizontal de l'ombre. Si la valeur passée est positive, l'ombre

sera projetée sur une distance égale à la valeur passée vers le bas en partant du bas de la boîte. Dans le cas contraire, l'ombre sera projetée vers le haut de la boîte en partant de son extrémité supérieure ;

4. **Le rayon de flou.** Cette valeur est encore une longueur et va nous permettre de rendre notre ombre floue en lui appliquant un flou Gaussien. La formule de création d'un flou Gaussien est assez complexe; reprenez simplement qu'une moyenne entre les couleurs des pixels environnants va être établie en mixant ces couleurs ensemble. Plus la valeur du flou va être grande, plus le flou de l'ombre sera diffus et l'ombre étalée ;
5. **Le rayon d'étendue de l'ombre.** Cette valeur est à nouveau une longueur et va servir à agrandir ou à rétrécir la taille de l'ombre. Par défaut, sa valeur est 0 et cela correspond à une ombre de même taille que la boîte. Passer une valeur positive augmentera d'autant la taille de l'ombre tandis que passer une valeur négative la rétrécira d'autant. C'est moins utilisé, mais sachez que ça existe ;
6. **La couleur de l'ombre.** Nous allons pouvoir définir la couleur de notre ombre. Toutes les valeurs de couleurs sont ici acceptées : nom de couleur, code hexadécimal, RGB, RGBa...

Les pseudo-classes

C'est un peu comme des classes mais répondant à des comportements particuliers de l'interface. Il y en a beaucoup, nous ne les verrons pas toutes... Certaines sont vraiment spécifiques et rarement utiles . Vous en trouverez la liste complète sur <https://developer.mozilla.org/fr/docs/Web/CSS> vous cliques sur pseudo classes dans la colonne de gauche et vous les verrez toutes.

Une pseudo classe s'écrit avec l'élément sur lequel il s'applique, suivi du symbole deux-points (:) pour finir par son nom.

:active

Quand vous cliquez sur un élément, celui ci devient actif. Cette pseudo-classe permet de donner un comportement particulier à cet élément cliqué.

```
div:active {  
    background-color: yellow;  
}
```

fera que la div qui sera cliquée passera en jaune au moment du clic. Notez que dès qu'on relâche le bouton, la div reprend sa couleur normale.

:hover

Va sélectionner l'élément que vous êtes en train de survoler avec la souris. Souvent utilisé avec les balises a, mais peut être utilisé avec n'importe quelle balise.

```
div:hover {  
    background-color: lime;  
}
```

mettra en vert clair la ou les div survolées. **Attention**, si vous avez une div dans une div, et que vous survolez la div enfant, le parent est survolé aussi.

:first-child

Va sélectionner le premier fils d'un élément donné. Par exemple, si vous avez :

```
<div>
  <p>Paragraphe 1</p>
  <p>Paragraphe 2</p>
</div>
```

et

```
div :first-child {
  background-color: lightblue;
}
```

Le premier paragraphe p aura la couleur bleu clair comme couleur de fond.

Attention à l'espace entre div et :first-child. On veut bien sélectionner les élément appartenant à la div...

:nth-child(an+b)

Ressemble beaucoup au précédent, mais au lieu de sélectionner le premier, on va lui indiquer quel(s) enfant(s) on veut sélectionner.

```
div :nth-child(3n) {
  background-color: pink;
}
```

va sélectionner un élément sur trois dans les fils des div à partir du premier (cela revient à $3n+0$).

```
div:nth-child(2n+1) {
  background-color: pink;
}
```

va sélectionner tous les éléments impairs. on peut aussi utiliser **even** et **odd** qui sont bien utiles pour sélectionner les éléments d'ordre pair ou impair, respectivement.

:last-child

Idem que :first-child, mais sélectionne le dernier fils du parent nommé.

:nth-last-child(an+b)

Idem que nth-child, mais en commençant par la fin

:first-of-type

Sélectionne le premier élément de chaque type rencontré dans l'élément nommé.

```
<div>
  un peu de <span>text</span>
  <p>et un paragraphe</p>
  <p>un second paragraphe</p>
```

```
<div>puis un bloc</div>
<div>et un second bloc</div>
  et <span>finalement</span> encore un peu de texte
</div>
```

avec le CSS

```
div :first-of-type {
  background-color: purple;
}
```

produira le résultat suivant

un peu de **texte**
et un paragraphe
un second paragraphe
puis un bloc
et un second bloc
et finalement encore un peu de texte

:nth-of-type(an+b)

Idem mais en indiquant quel(s) élément(s) on veut sélectionner.

:last-of-type

dernier élément de chaque type rencontrée

:nth-last-of-type(an+b)

Idem mais en indiquant quel(s) élément(s) on veut sélectionner à partir de la fin.